PARETOBANDIT: BUDGET-PACED ADAPTIVE ROUTING FOR NON-STATIONARY LLM SERVING

Annette Taberner-Miller [1]

ABSTRACT

Multi-model LLM serving operates in a non-stationary, noisy environment: providers revise pricing, model quality can shift or regress without notice, and new models arrive regularly. More than a dozen recent methods have proposed learned routers to navigate the resulting quality–cost tradeoff across portfolios spanning a $\sim 530\times$ cost range. Despite this activity, two gaps in the current solution space limit routing effectiveness under these conditions: no existing router enforces a dollar-denominated cost ceiling in closed loop over an open-ended request stream, and none provides principled online adaptation to post-deployment shifts in pricing or model quality. We present **ParetoBandit**, an open-source adaptive router built on cost-aware contextual bandits that addresses both gaps. Its core contributions are: (1) an online primal–dual budget pacer that enforces a per-request cost ceiling without a known horizon, and (2) geometric forgetting on sufficient statistics that gives the bandit bounded memory for tracking quality and cost shifts. A hot-swap model registry further supports runtime model changes with budget-controlled exploration. On 1,824 benchmark prompts with a three-model portfolio, the router maintains budget compliance within 0.4%, adapts to price and quality shifts with up to $+0.071$ quality lift, and integrates a cold-started model within ~ 142 steps.

Code: `https://github.com/ParetoBandit/ParetoBandit` | `pip install paretobandit`

1 INTRODUCTION

Production LLM serving relies on multi-model portfolios spanning a $\sim 530\times$ cost range, yet no single model dominates on every input. This complementarity has motivated a growing line of work on learned routers that dispatch each request to the most suitable model (Chen et al., 2024a; Ong et al., 2025; Chen et al., 2024b; Feng et al., 2025; Mei et al., 2025; Panda et al., 2025; Bhatti et al., 2026). The open question is no longer *whether* to route, but how to do so under the operational realities of deployment: dollar budgets, non-stationary pricing, and evolving model portfolios.

Many existing routers learn a fixed policy offline and freeze it at serving time, whether through cascading (Chen et al., 2024a), supervised classification on preference or embedding data (Ong et al., 2025; Chen et al., 2024b; Feng et al., 2025), or constrained optimization with budget targets (Mei et al., 2025; Bhatti et al., 2026). All navigate the quality–cost tradeoff, but none demonstrate adaptation to post-deployment shifts—despite the environment being non-stationary: providers silently update APIs in ways that shift quality (Chen et al., 2023; Ma et al., 2024), pricing can shift substantially (for instance, in 2024 OpenAI cut GPT-4o input prices by roughly 50% (OpenAI, 2024)), and new models launch regularly. PROTEUS (Bhatti et al., 2026) explicitly highlights cost-adaptive routing as an open problem.

Online learning offers a path forward. Bandit-based routers update their policy from the outcome of each dispatched request (Panda et al., 2025; Wang et al., 2025a; Li, 2025), and continual-learning frameworks further accommodate evolving model pools (Wang et al., 2025b). Yet even among these adaptive methods, two deployment gaps remain, which we detail below.

Challenge 1: Closed-loop enforcement of cost ceilings. Several routers incorporate budget or SLA targets (Panda et al., 2025; Mei et al., 2025; Bhatti et al., 2026), each tailored to a specific deployment model: PILOT (Panda et al., 2025) operates over a known finite horizon, OmniRouter (Mei et al., 2025) optimizes over a fixed prompt set, and PROTEUS (Bhatti et al., 2026) fixes its dual variables at deployment time. Closed-loop enforcement of a per-request cost ceiling over an open-ended request stream—where the horizon is unknown and conditions may shift—remains an open problem.

Challenge 2: Principled handling of non-stationarity. Non-stationary bandit algorithms are well studied (Garivier & Moulines, 2011; Russac et al., 2019). However, we are not aware of any published LLM router that (i) explicitly employs geometric discounting or sliding-window mechanisms

[1]Independent Researcher. Correspondence to: Annette Taberner-Miller <taberner@alum.mit.edu>.

to handle post-deployment shifts, or (ii) systematically evaluates routing behaviour under model quality degradation or changes in model pricing during deployment. In our quality-degradation experiment (Section 4.4), a bandit baseline without closed-loop budget control detects the degraded model but overshoots the cost target by up to $6.9\times$.

We address these challenges with **ParetoBandit**, an open-source adaptive routing system for budget-constrained, non-stationary LLM serving. ParetoBandit builds on cost-aware contextual bandits and introduces three system-level contributions:

1. **Autonomous budget control** (Section 3.2). An online primal–dual budget pacer accepts an operator-specified per-request cost ceiling and enforces it in closed loop over an open-ended request stream with no known horizon, automatically reallocating freed budget when prices drop.
2. **Non-stationarity resilience** (Section 3.3). Geometric forgetting gives the bandit a bounded effective memory that tracks shifts in both quality and cost. Exploration and forgetting rates are chosen via a Pareto knee-point procedure (Appendix A) anchored by a single practitioner-specified adaptation horizon.
3. **Runtime portfolio management** (Section 3.6). A hot-swap model registry supports adding and removing models at runtime. A brief forced-exploration phase bootstraps each newcomer's posterior, after which UCB selection discovers its quality–cost niche while the budget pacer maintains compliance. The router discriminates rather than blindly adopting (Section 4.5).

We evaluate ParetoBandit on a 1,824-prompt holdout set drawn from nine public benchmarks, using a three-tier model portfolio ($K{=}3$, with a $K{=}4$ onboarding extension) and an LLM judge with a continuous rubric (Section 4.1). Section 3 details the system design, and Section 4 evaluates it with respect to both deployment challenges.

2 Background and Problem

We formalize LLM routing as an online decision-making problem in which the router learns which model to dispatch for each prompt, respects an operator-specified cost ceiling, and adapts when model quality or pricing shifts after deployment. This section defines the three components of the problem; Section 3 presents our solution.

2.1 Contextual Bandit Routing

At each discrete step $t = 1, 2, \ldots$, a user prompt arrives and the router observes a context vector $x_t \in \mathbb{R}^d$ derived from the prompt (Section 2.2). The router selects a single model a_t from a portfolio $\mathcal{A} = \{1, \ldots, K\}$ and dispatches the prompt. After inference, the system observes a scalar reward r_t reflecting response quality and a realized cost c_t in dollars. The reward may be derived from an automated LLM judge, explicit user feedback, or a task-completion metric; in our setup it is a continuous score normalized to $[0, 1]$ by the judge rubric (Section 4.1). The objective is to learn a policy π that maximizes cumulative quality subject to a ceiling on average cost per request (detailed in Section 2.3):

$$\max_{\pi} \ \mathbb{E}\left[\sum_{t=1}^{T} r_{t,\,\pi(x_t)}\right] \qquad (1)$$
$$\text{s.t.} \quad \limsup_{T\to\infty} \ \tfrac{1}{T}\textstyle\sum_{t=1}^{T} c_{\pi(x_t)} \le B$$

where B is the operator's maximum acceptable average cost per request in dollars (e.g., \$0.005/req). The router may spend well below B when cheaper models deliver comparable quality; it approaches the ceiling only when doing so yields a meaningful quality gain. We relax this via a per-request Lagrange multiplier λ_t whose scoring function and dynamics are detailed in Section 3.2.

The router operates under *bandit feedback* (Lattimore & Szepesvári, 2020): it observes the reward and cost only for the model to which a query was routed, never the counterfactual outcomes from the $K{-}1$ alternatives. This partial observability is inherent to LLM serving, where evaluating every model on every prompt would multiply cost by K.

2.2 Context Representation

The context vector x_t is constructed by encoding the prompt with all-MiniLM-L6-v2, a lightweight model from the Sentence Transformers framework (Reimers & Gurevych, 2019), projecting to 25 PCA components whitened to unit variance, and appending a bias term, yielding $d = 26$. The PCA projection is fitted on $\sim$46,000 disjoint LMSYS Arena prompts (Zheng et al., 2023). Section 3.5 details the latency impact of this dimensionality choice.

2.3 Budget Constraint

Each model $a \in \mathcal{A}$ incurs a deterministic cost $c_a(x_t)$ per request, typically computed from per-token API pricing. In a realistic portfolio, costs span orders of magnitude: the three-tier pool used in our experiments ranges from $\sim$\$0.00003/req (Llama-3.1-8B) to $\sim$\$0.015/req (Gemini-2.5-Pro), a $530\times$ spread.

The rate constraint in Eq. 1 departs from the classical Bandits with Knapsacks (BwK) framework (Badanidiyuru et al., 2013), which imposes a cumulative budget over a known horizon T. Production LLM serving is open-ended: request volume is unknown, and the natural specification is a cost *rate* (dollars per request), not a finite pool. Our formulation is horizon-free and self-regulates at any traffic volume.

We enforce the rate constraint through a dual variable λ_t that rises when recent spending exceeds B and falls when it is below. Section 3.2 defines the smoothed dual-ascent update and the two-layer enforcement mechanism (soft penalty plus hard cost ceiling) that operationalize λ_t.

2.4 Non-Stationarity

We address two forms of environment drift that affect production LLM deployments. *Reward drift* occurs when providers silently update models behind their APIs (Chen et al., 2023; Ma et al., 2024), changing the conditional expected reward. *Cost drift* occurs when API pricing changes significantly (OpenAI, 2024). The two interact: a price drop changes which models are cost-effective, requiring the router to jointly re-learn reward estimates and re-optimize the quality–cost frontier. Section 3.3 presents our approach using geometric discounting (Garivier & Moulines, 2011; Russac et al., 2019).

3 System Design

ParetoBandit jointly learns reward surfaces from bandit feedback, enforces a long-run budget constraint ceiling, and adapts to non-stationarity, a combination that, to our knowledge, ParetoBandit is the first LLM routing system to integrate. It composes four foundational ingredients: LinUCB (Li et al., 2010) for contextual reward learning, BwK-inspired Lagrangian relaxation (Badanidiyuru et al., 2013; Agrawal et al., 2016) for budget enforcement, geometric discounting (Garivier & Moulines, 2011; Russac et al., 2019) for non-stationarity, and offline-to-online warm-start priors (Oetomo et al., 2023) for cold-start elimination. Their composition rests on LinUCB's sufficient-statistic representation (A_a, b_a per arm): forgetting reduces to a scalar multiply, warmup to a matrix addition, and updates to $O(d^2)$ Sherman–Morrison operations, while the confidence bonus $x_t^\top A_a^{-1} x_t$, the exact posterior variance under a Gaussian linear model, provides context-dependent calibrated exploration from the first request. We chose UCB over Thompson Sampling (Agrawal & Goyal, 2013) because its deterministic score interacts more predictably with the Lagrangian penalty.

Although each ingredient is well-established individually, combining them creates cross-component interactions. For example, geometric forgetting shrinks A_a toward singularity over time, inflating LinUCB's confidence bonus and making expensive but uncertain arms more attractive. The BudgetPacer's dual variable λ_t compensates online (Section 3.2), but the upstream interaction between the forgetting rate γ and exploration coefficient α determines exploration aggressiveness; these should be calibrated jointly, not in isolation. Appendix A describes the Pareto knee-point procedure that resolves this.

3.1 Architecture Overview

ParetoBandit operates as a closed-loop routing system with two data paths.

Synchronous inference path. When a user prompt arrives, the feature extractor (Section 2.2) produces a PCA-reduced context vector x_t. The `BanditRouter` scores every eligible model using the budget-augmented UCB rule (Section 3.2) and dispatches the prompt to the highest-scoring model a_t.

Asynchronous feedback path. After inference, the observed reward r_t and realized cost c_t flow back. The bandit updates its per-arm sufficient statistics with geometric forgetting (Section 3.3), while the budget pacer updates λ_t via the EMA-smoothed cost signal (Eqs. 3–4). The context vector is cached at route time so that rewards arriving hours later (e.g., human Reinforcement Learning from Human Feedback (RLHF) labels) can still update the bandit without re-encoding the prompt. Algorithm 1 consolidates the full per-step loop.

3.2 Budget-Paced Arm Selection

At each step t, the router selects model a_t by maximizing a budget-augmented utility:

$$a_t = \arg\max_{a \in \mathcal{A}_t} \Big[\underbrace{\hat{\theta}_a^\top x_t}_{\text{exploit}} + \underbrace{\alpha \sqrt{x_t^\top A_a^{-1} x_t}}_{\text{explore}} - \underbrace{(\lambda_c + \lambda_t)\, \tilde{c}_a}_{\text{cost penalty}} \Big] \tag{2}$$

The underbraced terms trade off exploitation of the current reward estimate, uncertainty-driven exploration (further inflated for stale arms via Eq. 9), and a cost penalty that discourages expensive models. The exploration rate $\alpha > 0$ scales the confidence bonus directly. The cost penalty has two additive components: a static weight $\lambda_c \geq 0$ (default 0.3) encoding the operator's baseline cost–quality preference from the first request ($\lambda_c = 0$ recovers quality-only routing), and a dynamic dual variable $\lambda_t \geq 0$ that starts at zero and rises only when recent spending exceeds B, providing closed-loop enforcement. After each request, a smoothed dual-ascent step adjusts λ_t:

$$\bar{c}_t = (1 - \alpha_{\text{ema}})\, \bar{c}_{t-1} + \alpha_{\text{ema}}\, c_t \tag{3}$$

$$\lambda_{t+1} = \big[\lambda_t + \eta\, (\bar{c}_t / B - 1)\big]_0^{\bar{\lambda}} \tag{4}$$

where $\bar{c}_t$ is an EMA-smoothed cost signal (α_{ema}=0.05, half-life ≈14 requests), η=0.05 is the step size, and $[\cdot]_0^{\bar{\lambda}}$ projects onto $[0, \bar{\lambda}{=}5]$. The EMA prevents sawtooth oscillations from single expensive requests; normalizing the gradient by B makes η portfolio-independent. When $\bar{c}_t / B > 1$ (overspending), λ_t rises, penalizing expensive models; when below, λ_t falls, releasing the router to pursue quality.

Algorithm 1 ParetoBandit: Budget-Paced Non-Stationary Routing

Require: Arms $\mathcal{A}$; regularization λ_0; budget B; static cost penalty λ_c; exploration α; forgetting γ; step size η; EMA α_{ema}; cap $\bar{\lambda}$

Init: $\forall\, a$: $A_a \leftarrow \lambda_0 I$, $b_a \leftarrow \mathbf{0}$ (or warmup both, §3.4), $\hat{\theta}_a \leftarrow A_a^{-1} b_a$; $\lambda_t \leftarrow 0$; $\bar{c}_0 \leftarrow B$; $t \leftarrow 0$
for each request with context x_t **do**
  — Arm Selection —
  if $\lambda_t > 0$ **then**
    $\mathcal{A}_t \leftarrow \{a \in \mathcal{A} : c_a \le c_{\max}^{\mathcal{A}}/(1+\lambda_t)\}$ ▷ Hard ceiling
  else
    $\mathcal{A}_t \leftarrow \mathcal{A}$
  end if
  for each $a \in \mathcal{A}_t$ **do**
    $\text{dt}_a \leftarrow t - \max(\text{last_upd}_a, \text{last_play}_a)$ ▷ Exploration staleness
    $v_a \leftarrow x_t^\top A_a^{-1} x_t / \max(\gamma^{\text{dt}_a}, V_{\min}^{-1})$ ▷ Variance inflation
    $s_a \leftarrow \hat{\theta}_a^\top x_t + \alpha\sqrt{v_a} - (\lambda_c + \lambda_t)\,\tilde{c}_a$
  end for
  $a_t \leftarrow \arg\max_{a \in \mathcal{A}_t} s_a$ ▷ Random tiebreak
  $t \leftarrow t+1$; $\text{last_play}_{a_t} \leftarrow t$
  — Observe reward r_t, cost c_t **—**
  — Reward Update (Geometric Forgetting) —
  $\text{dt}' \leftarrow t - \text{last_upd}_{a_t}$ ▷ Decay staleness (statistics age)
  $A_{a_t} \leftarrow \gamma^{\text{dt}'} A_{a_t}$; $b_{a_t} \leftarrow \gamma^{\text{dt}'} b_{a_t}$ ▷ Decay stale data
  $A_{a_t}^{-1} \leftarrow A_{a_t}^{-1}/\gamma^{\text{dt}'}$ ▷ $O(d^2)$ scalar op
  $A_{a_t} \leftarrow A_{a_t} + x_t x_t^\top$; $b_{a_t} \leftarrow b_{a_t} + r_t\, x_t$
  Update $A_{a_t}^{-1}$ via Sherman–Morrison ▷ $O(d^2)$
  $\hat{\theta}_{a_t} \leftarrow A_{a_t}^{-1} b_{a_t}$; $\text{last_upd}_{a_t} \leftarrow t$
  — Budget Pacer Dual Update —
  $\bar{c}_t \leftarrow (1-\alpha_{\text{ema}})\,\bar{c}_{t-1} + \alpha_{\text{ema}}\, c_t$ ▷ Eq. 3
  $\lambda_{t+1} \leftarrow [\lambda_t + \eta(\bar{c}_t/B - 1)]_0^{\bar{\lambda}}$ ▷ Eq. 4
end for

Reward model and parameter estimation. LinUCB models the expected reward as $\mathbb{E}[r_t \mid x_t, a] = \theta_a^{*\top} x_t$ (Li et al., 2010). The estimate $\hat{\theta}_a = A_a^{-1} b_a$ is the ridge regression solution, where

$$A_a = \lambda_0 I + \textstyle\sum_{\tau:\, a_\tau = a} x_\tau x_\tau^\top,\; b_a = \sum_{\tau:\, a_\tau = a} r_\tau\, x_\tau \quad (5)$$

are the per-arm *sufficient statistics*, the running aggregates that fully summarize past observations for parameter estimation, eliminating the need to store raw history. At cold start ($A_a = \lambda_0 I$), the confidence bonus is maximal, driving exploration; as observations accumulate, the confidence set shrinks toward exploitation.

Log-normalized cost. The term $\tilde{c}_a \in [0,1]$ is a log-normalized unit cost:

$$\tilde{c}_a = \frac{\log c_a - \log c_{\text{floor}}}{\log c_{\text{ceil}} - \log c_{\text{floor}}} \quad (6)$$

where c_{floor}=\$0.0001 and c_{ceil}=\$0.10 per 1k tokens are fixed market bounds. The logarithmic scale compresses the $530\times$ cost range into a bounded penalty commensurate with the $[0,1]$ reward scale (validated in Appendix B).

Two-layer enforcement (Adaptive mode). The budget pacer combines two mechanisms simultaneously:

- **Hard ceiling (safety net):** When $\lambda_t > 0$, the candidate set $\mathcal{A}_t$ excludes models whose blended price exceeds a dynamic ceiling $c_{\max}^{\mathcal{A}}/(1+\lambda_t)$, where $c_{\max}^{\mathcal{A}} = \max_{a \in \mathcal{A}} c_a$ is the portfolio's most expensive rate. This acts as a circuit breaker, preventing catastrophic overspend on any single request.
- **Soft penalty (optimizer):** The dual penalty $\lambda_t \tilde{c}_a$ biases the UCB score toward cheaper models proportionally to their cost, allowing fine-grained, context-dependent routing within the surviving candidate set.

The hard ceiling provides per-request worst-case cost bounds independent of λ_t; the soft penalty provides smooth, quality-maximizing allocation within those bounds.

Theoretical status. Eq. 2 is a Lagrangian relaxation of Eq. 1, but four departures from the classical BwK analysis (Badanidiyuru et al., 2013)—rate constraint, EMA smoothing, hard ceiling, and log-normalized costs—void the formal $\tilde{O}(\sqrt{T})$ bounds. The smoothed dual framework of Balseiro & Gur (2019) accommodates EMA-like averaging but assumes i.i.d. contexts and a convex program structure; our non-stationary setting with geometric forgetting violates both conditions, so their formal guarantees do not apply either. Three properties nonetheless hold by construction: (1) $\lambda_t \in [0, \bar{\lambda}]$, (2) staleness inflation ensures re-exploration of neglected arms, and (3) when $\lambda_t > 0$, the hard ceiling caps the cost of any single request. Budget compliance is therefore validated empirically: across all conditions in Section 4, per-request cost never exceeds the ceiling by more than $\sim$4%.

3.3 Non-Stationary Adaptation via Geometric Forgetting

Geometric forgetting enables ParetoBandit to override stale estimates within its effective memory window, whereas a standard LinUCB (γ=1.0) accumulates all historical data equally and may require hundreds or thousands of new observations to do the same. Concretely, ParetoBandit applies a forgetting factor $\gamma \in (0,1]$ directly to the per-arm sufficient statistics. Before incorporating the new observation

(x_t, a_t, r_t), the design matrix and reward accumulator for the chosen arm a_t are exponentially discounted:

$$A_{a_t} \leftarrow \gamma^{\mathrm{dt}} A_{a_t} + x_t x_t^\top \tag{7}$$

$$b_{a_t} \leftarrow \gamma^{\mathrm{dt}} b_{a_t} + r_t x_t \tag{8}$$

where $\mathrm{dt} = t - \text{last_update}_{a_t}$ is the number of intervening requests since a_t's last update. This batched exponentiation avoids redundant per-step multiplications for arms that were not selected. The exponential weighting (Garivier & Moulines, 2011) gives observations an effective half-life of $\ln 2/(1-\gamma)$ steps and an e-folding time of $1/(1-\gamma)$. For γ=0.997, the e-folding time is ≈333 steps: an observation from 333 steps ago retains weight $\gamma^{333} \approx e^{-1} \approx 0.37$, and after ∼1,000 steps the prior's contribution is reduced to $\gamma^{1000} \approx 0.05$.

Staleness-based variance inflation. Geometric forgetting passively increases $x_t^\top A_a^{-1} x_t$ for idle arms, but this may be too slow when the environment shifts. The router explicitly inflates the UCB variance using the *exploration staleness* $\mathrm{dt}_a = t - \max(\text{last_update}_a, \text{last_played}_a)$, which counts from whichever happened more recently—a statistics update or a play—so that arms dispatched but still awaiting asynchronous rewards are not prematurely re-explored:

$$v_a = \frac{x_t^\top A_a^{-1} x_t}{\max\left(\gamma^{\mathrm{dt}_a},\ V_{\max}^{-1}\right)} \tag{9}$$

Without a cap, the inflation grows without bound as $\mathrm{dt}_a \to \infty$, and the exploration bonus $\alpha\sqrt{v_a}$ eventually dominates any finite cost penalty, forcing selection of stale expensive arms regardless of budget pressure. A capacity cap $V_{\max} = 200$ bounds the worst-case inflation to $\sqrt{V_{\max}} \approx 14\times$ the uninflated bonus, large enough to reliably trigger re-exploration yet bounded enough that the cost signal retains meaningful influence on arm selection.

Interaction with the budget pacer. Geometric forgetting interacts with budget pacing through a feedback loop. As γ shrinks A_a over time, the confidence bonus grows, increasing the probability of selecting expensive arms that happen to be uncertain. This drives up λ_t, which penalizes those arms, partially counteracting the inflation. The EMA smoothing in the dual update (Eq. 3) further dampens the resulting oscillations. The parameter γ thus controls a bias-variance tradeoff between *stability* (trusting historical evidence) and *plasticity* (responding quickly to drift). The forgetting rate is therefore selected jointly with α (Appendix A).

3.4 Offline-to-Online Warmup Priors

Without warmup priors, the router begins with uninformative $A_a = \lambda_0 I$, $b_a = \mathbf{0}$, and needs to explore each model on diverse prompts before learning which model excels where. ParetoBandit eliminates this cold start by loading offline sufficient statistics $(A_a^{\mathrm{off}}, b_a^{\mathrm{off}})$ fitted on historical prompt-reward data. A tunable prior strength n_{eff} controls how many pseudo-observations the offline data contribute:

$$s = \frac{n_{\mathit{eff}}}{A_a^{\mathrm{off}}[d,d]} \tag{10}$$

where $A_a^{\mathrm{off}}[d,d]$ is the total precision mass in the bias direction. The scaled prior is regularized with a mean-preserving correction:

$$A_a \leftarrow s \cdot A_a^{\mathrm{off}} + \lambda_0 I \tag{11}$$

$$b_a \leftarrow s \cdot b_a^{\mathrm{off}} + \lambda_0\, \hat{\theta}_a^{\mathrm{off}} \tag{12}$$

The $\lambda_0 \hat{\theta}_a^{\mathrm{off}}$ term in b_a prevents $\lambda_0 I$ regularization from shrinking the posterior mean toward zero, ensuring $A_a^{-1} b_a \approx \hat{\theta}_a^{\mathrm{off}}$ at the desired confidence level. For models absent from the offline data, a heuristic prior places n_{eff} pseudo-observations at isotropic uncertainty with a bias-only reward prediction.

Geometric forgetting ensures priors decay naturally (Section 3.3), so steady-state quality is determined by online evidence regardless of initialization. Priors are beneficial in deployment comparisons but not necessary—cold-start convergence is demonstrated in Section 4.5 and ablated in Appendices C–D.

3.5 Computational Efficiency

The router sits on the critical path of every LLM request. ParetoBandit maintains a cached A_a^{-1} and applies Sherman–Morrison rank-1 updates in $O(d^2)$; geometric discounting adds only a scalar division $A_a^{-1} \leftarrow A_a^{-1}/\gamma$. Arm selection requires one $O(d^2)$ quadratic form per arm for the UCB bonus plus an $O(d)$ dot product for the reward estimate. With K=3 and d=26, the full route-and-update cycle completes in 43 μs (p50), sustaining ∼22,000 req/s on a single CPU core. Including prompt embedding and PCA projection, end-to-end routing latency is 9.8 ms—under 1% of a typical ∼1 s inference call (Ganglani, 2026). PCA reduction and inverse caching together yield a ∼44× throughput gain over a raw-dimension baseline (Appendix F).

3.6 Runtime Portfolio Management

Production model portfolios are not static. ParetoBandit supports runtime model additions and removals via `add_arm()` and `delete_arm()` without restarting the router. A newly added model is initialized with either a heuristic prior or uninformative covariance. When existing arms have strong learned posteriors and α is small, the UCB exploration bonus alone may be insufficient to trigger natural exploration of a cold-start arm. Section 4.5 addresses

this with a short forced-exploration burn-in (20 pulls routed unconditionally to the new arm), after which the bandit has enough evidence to discriminate and UCB selection takes over. The context vector x_t is cached at route time so that feedback arriving later (synchronous judge scores, asynchronous RLHF labels, or batch metrics) can update the bandit without re-encoding the prompt, with both in-memory and SQLite-backed storage backends.

4 EVALUATION

We evaluate ParetoBandit across four experiments: stationary budget pacing, budget pacing under cost drift, resilience to silent quality degradation, and cold-start model onboarding. All experiments use 20 seeds and report 95% bootstrap confidence intervals.

4.1 Experimental Setup

Data and models. We collect 11,983 prompts from nine public NLP benchmarks (MMLU (Hendrycks et al., 2021), GSM8K (Cobbe et al., 2021), HellaSwag (Zellers et al., 2019), BIG-Bench Hard (Suzgun et al., 2023), ARC-Challenge (Clark et al., 2018), OpenBookQA (Mihaylov et al., 2018), WinoGrande (Sakaguchi et al., 2020), TruthfulQA (Lin et al., 2022), MBPP (Austin et al., 2021)) spanning reasoning, math, code synthesis, commonsense, and factual knowledge. For each prompt, all K=3 models generate responses that are judged offline, producing a full reward–cost matrix. Prompts are partitioned into three disjoint splits stratified by source: train (n=8,374) for fitting warmup priors, val (n=1,785) for online hyperparameter tuning, and test (n=1,824) for evaluation. The K=3 portfolio spans a 530× cost range (Table 1).

Quality evaluation. Each response is scored by DeepSeek-R1 (DeepSeek-AI, 2025) on three continuous dimensions: reasoning quality (weight 0.4), instruction following (0.3), and communication quality (0.3), yielding a composite reward $r_t \in [0, 1]$. R1 produces the largest inter-model reward gaps among the judges we evaluated; Appendix E validates that its routing decisions capture $\geq$97% of any alternative judge's oracle reward.

Baselines. Two conditions represent increasing routing sophistication, all sharing warmup priors (n_{eff}=1164, α=0.01):

1. **Naive Bandit** (γ=1.0)**:** LinUCB with online learning but infinite memory and a static cost penalty.
2. **ParetoBandit** (γ=0.997)**:** Geometric forgetting with warmup priors and an active BudgetPacer.

Experiment 2 additionally includes a **Recalibrated Bandit** (oracle knowledge of price changes) and a **Forgetting Bandit** (γ=0.997, no pacer) as ablations. Hyperparameters are selected via the Pareto knee-point procedure described in Appendix A.

Non-stationary protocol. Experiments 2–3 follow a three-phase stress-test: normal operation (608 prompts), abrupt perturbation (608 prompts), and recovery (608 prompts). Phase 3 reuses Phase 1 prompts for a controlled within-subject comparison.

Table 1. Model portfolio and budget targets. The 530× cost spread stresses the router's quality–cost tradeoff. Three budgets span the full operating range.

Model	**Tier**	**Cost ($/req)**
Llama-3.1-8B	Budget	$\$2.9\times10^{-5}$
Mistral-Large	Mid-cost	$\$5.3\times10^{-4}$
Gemini-2.5-Pro	Frontier	$\$1.5\times10^{-2}$
Budget	**Target B**	**Regime**
Tight	$\$3.0\times10^{-4}$	Llama-dominant
Moderate	$\$6.6\times10^{-4}$	Llama–Mistral mix
Loose	$\$1.9\times10^{-3}$	Selective Gemini

4.2 Stationary Budget Pacing

We first ask: *can the router accept a dollar budget ceiling and automatically maximize quality beneath it, filling the quality–cost gaps between fixed single-model baselines?*

Figure 1a shows the quality–cost Pareto frontier on the test set. Each fixed model occupies a single point on this frontier, plotted as (mean cost per request, mean quality score): Llama at ($\$2.9\times10^{-5}$, 0.793), Mistral at ($\$5.3\times10^{-4}$, 0.923), Gemini at ($\$1.5\times10^{-2}$, 0.932). The BudgetPacer traces a continuous curve through these points by dynamically mixing models based on prompt context and budget pressure. At a budget of $\$2.3\times10^{-4}$/req, the router achieves 92% [92, 92] of Gemini quality at 2% of its cost by blending 56% Llama and 44% Mistral. For binding ceilings, utilisation ranges from 0.98× to 1.00× (Figure 1b). When the ceiling is high enough that cost never binds, the cost penalty λ_t decays to zero and the router reduces to pure quality maximization, recovering 96.4% [96.4, 96.5] of an oracle that selects the highest-scoring model for each prompt (0.963). In practice, this turns model selection from a discrete choice among K fixed operating points into a continuous budget dial: the operator sets a dollar ceiling, and the router discovers the best quality mix beneath it (Figure 1c).

4.3 Budget Pacing under Cost Drift

We now evaluate whether the pacer automatically exploits a mid-stream price drop while maintaining compliance. Phase 1 uses normal pricing; Phase 2 drops Gemini-2.5-Pro's pricing to $0.10/M tokens (normalized cost $\tilde{c} \approx 0$);

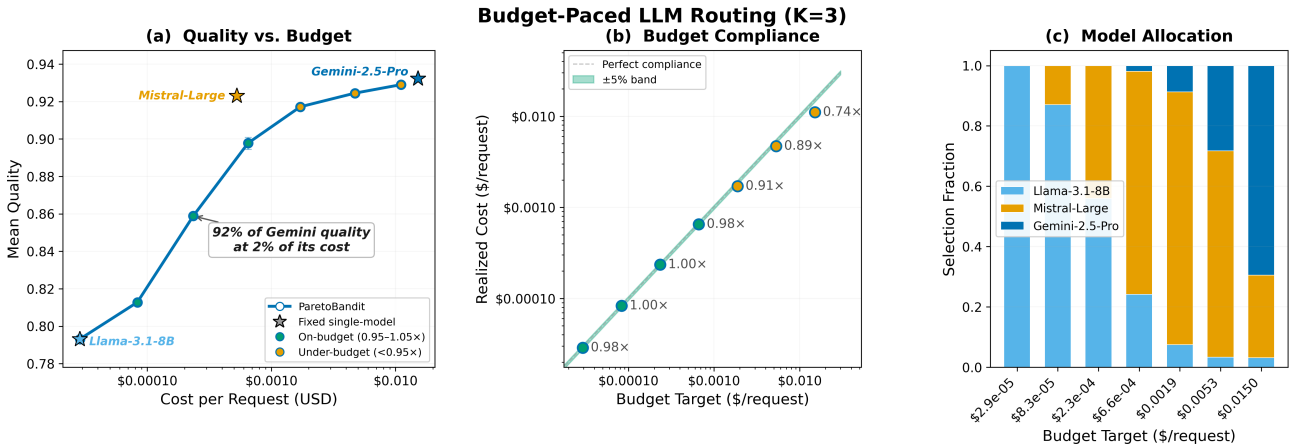


Figure 1. Quality–cost Pareto frontier under stationary conditions (K=3, 20 seeds). **(a)** ParetoBandit with BudgetPacer (blue curve) traces a continuous quality–cost frontier by accepting a dollar budget ceiling. Fixed single-model baselines shown as stars. **(b)** Budget compliance: realized cost vs. ceiling. Green band marks $\pm 5\%$. **(c)** Model allocation shifts from Llama-dominant at tight budgets to Gemini-heavy at loose budgets.

Table 2. Budget compliance under cost drift (20 seeds). Each cell shows realised cost as a multiple of the ceiling (1.00× = at ceiling); 95% bootstrap CIs are reported inline for key claims in the text. **Bold**: within 5% of ceiling.

Budget	Condition	Phase 1	Phase 2	Phase 3
Tight ($\$3.0\times10^{-4}$)	Naive Bandit	**1.07**×	0.89×	1.09×
	Recalibrated	**1.07**×	**0.99**×	1.44×
	Forgetting Bandit	2.62×	0.88×	5.50×
	ParetoBandit	**1.01**×	0.95×	**1.04**×
Moderate ($\$6.6\times10^{-4}$)	Naive Bandit	0.57×	0.40×	0.61×
	Recalibrated	0.57×	0.44×	2.14×
	Forgetting Bandit	1.50×	0.43×	4.13×
	ParetoBandit	**0.99**×	0.65×	**1.00**×
Loose ($\$1.9\times10^{-3}$)	Naive Bandit	3.07×	0.14×	3.26×
	Recalibrated	3.07×	0.17×	0.94×
	Forgetting Bandit	3.08×	0.17×	4.93×
	ParetoBandit	0.96×	0.23×	0.95×

Phase 3 restores original pricing.

Budget compliance is the key differentiator. Table 2 summarises cost/ceiling ratios across all conditions and phases. ParetoBandit is the only condition that reliably stays within the ceiling in Phase 1 (0.96× [0.95, 0.96] to 1.01× [1.00, 1.01]) and recovers compliance in Phase 3 (0.95× [0.92, 0.97] to 1.04× [1.01, 1.09]). The critical ablation is the Forgetting Bandit (γ=0.997, no pacer), which shares ParetoBandit's learning dynamics yet produces consistently poor compliance (2.62× [2.34, 2.97] at tight Phase 1, 5.50× [4.99, 6.05] at tight Phase 3), confirming that the BudgetPacer—not the forgetting factor—drives budget control.

Exploiting the price drop. When Gemini becomes nearly free in Phase 2, the BudgetPacer detects the cost change via its EMA signal. All three budget regimes show significant reward lifts (tight $\Delta = +0.071$ [0.068, 0.074]; loose $\Delta = +0.018$ [0.015, 0.021]), with the tight budget exhibiting the largest gain because its Phase 1 constraint was most binding. Figure 2 shows the dual-variable dynamics: λ_t decays as costs fall below the ceiling, increasing Gemini adoption; in Phase 3, λ_t rises and compliance recovers. This full round-trip—exploit, then recover—demonstrates bidirectional adaptation without operator intervention.

4.4 Silent Quality Degradation

We stress-test against a silent quality regression: Mistral-Large's reward drops to 0.75 (∼18% below normal) while its API continues to respond and charge at normal rates. The cost signal provides no warning; only the reward reveals the problem. Phase 3 restores normal quality, testing whether the router re-discovers the recovered model.

Results (Figure 3). ParetoBandit detects the quality drop purely through the reward signal, reducing Mistral allocation from 71% to 50% in Phase 2 at moderate budget. In Phase 3, geometric forgetting down-weights the corrupted estimates and staleness-driven exploration begins re-exploring Mistral, raising allocation to 54% and recovering Phase 3 reward to 0.8630 [0.8572, 0.8684] (vs. Phase 1 0.8850 [0.8806, 0.8891], recovery ratio 0.975 [0.967, 0.982]). Budget compliance holds throughout (0.95× [0.95, 0.96] to 1.00× [0.99, 1.01] across all nine phase–budget cells). The unconstrained baseline is unaffected (Phase 2 reward 0.9263 [0.9253, 0.9271] vs. Phase 1 0.9245 [0.9235, 0.9255]) but incurs a 24.2% cost increase from over-allocating to Gemini. This ∼18% degradation lies within the system's recovery envelope; Appendix G



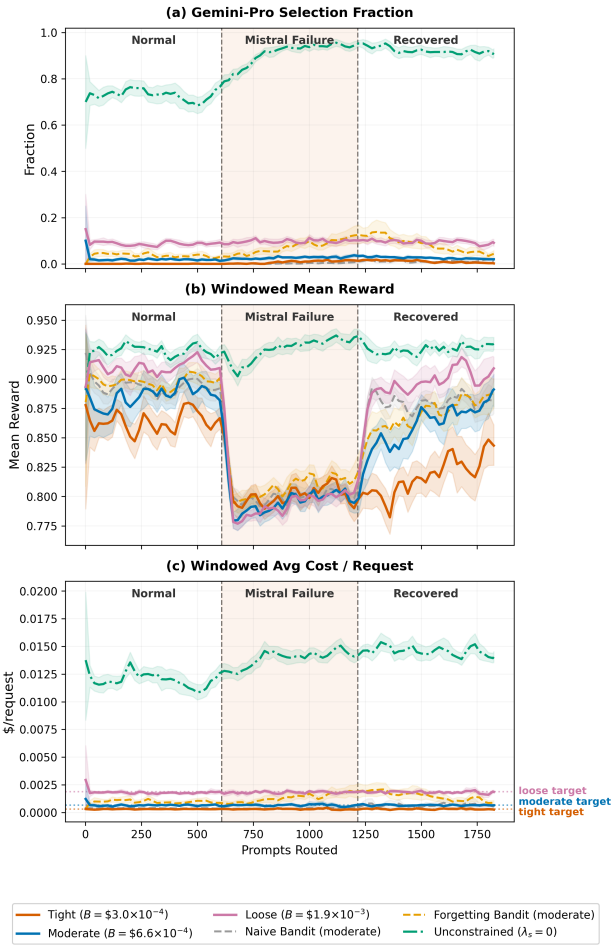


Figure 2. Adaptation dynamics under cost drift (K=3, 20 seeds, 95% bootstrap CI). Normal pricing → Gemini price drop at prompt 608 → price restored at prompt 2×608. **(a)** Gemini-Pro selection fraction: all budget tiers surge toward Gemini during the price drop, then revert when pricing is restored. **(b)** Windowed mean reward: all conditions improve during Phase 2 as Gemini becomes budget-accessible. **(c)** Windowed average cost per request (dotted = budget ceilings): ParetoBandit tracks the ceiling in all three phases; the Forgetting Bandit overshoots from Phase 1 onward.

Figure 3. Silent quality degradation dynamics (20 seeds, 95% bootstrap CI). Mistral-Large degrades at prompt 608 and recovers at 2×608; cost is unchanged. **(a)** Gemini-Pro selection fraction rises modestly under budget constraints and sharply (∼17 pp) without one. **(b)** Windowed reward: loose and unconstrained conditions recover fully in Phase 3; tighter budgets are still converging at the horizon limit (Appendix G). **(c)** Cost per request (dotted lines = ceilings): ParetoBandit holds compliance throughout, confirming that the degradation is invisible to the cost signal.

characterises the threshold beyond which recovery requires an extended horizon.

4.5 Cold-Start Model Onboarding

After Phase 1 learning on the K=3 portfolio, Gemini-2.5-Flash is added as a fourth arm via `register_model()` with no warmup priors. Figure 4 shows Flash adoption across three scenario–budget combinations. In the Good & Cheap scenario (Figure 4a), all 80 trials (20 seeds × 4 budget tiers) achieve sustained adoption within ∼142 steps. Budget determines the equilibrium share, not whether adoption occurs: loose budgets settle at ∼10.2% [7.4, 13.1] Flash share while tight budgets plateau at ∼4.4% [3.6, 5.2]. The BudgetPacer maintains compliance throughout the K=3 → K=4 transition without reconfiguration (Figure 5). Crucially, the bandit discriminates rather than blindly adopting: expensive models are budget-gated (Figure 4b) and bad models are rejected after a bounded burn-in (Figure 4c).

The trade-off relative to offline characterisation is that ∼1% of Phase 2 requests serve as online exploration, but adoption is automatically budget-gated and requires no paired preference data.

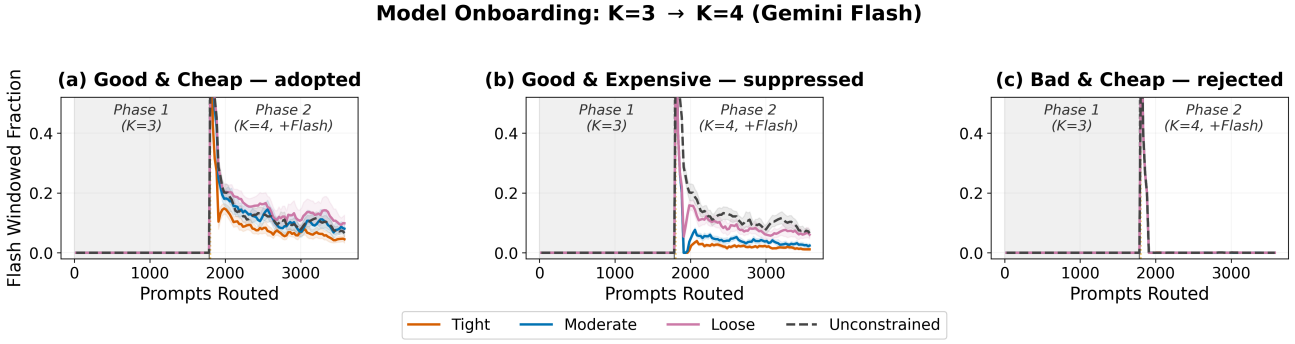


Figure 4. Model onboarding ($K{=}3 \to K{=}4$; 20 seeds, 95% bootstrap CI). Flash windowed selection fraction across four budget levels after cold-start addition. **(a)** Good & cheap: Flash is adopted at all budgets. **(b)** Good & expensive: the BudgetPacer suppresses Flash under tight budgets. **(c)** Bad & cheap: the bandit correctly rejects Flash in every seed.

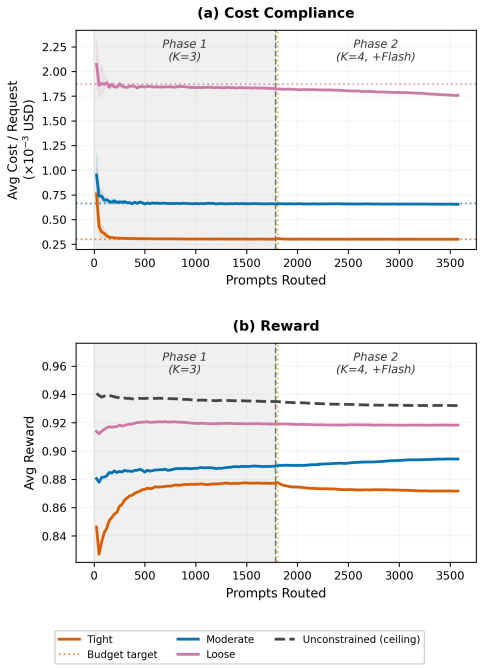


Figure 5. Budget–quality trade-off during model onboarding (Good & Cheap; 20 seeds, 95% bootstrap CI). **(a)** Running cost per request (dotted = targets). Tight and moderate track their ceilings through the $K{=}3 \to K{=}4$ transition; loose settles below target as its constraint is only weakly binding. **(b)** Cumulative reward. Tight dips briefly at the Phase 2 boundary (20 forced-exploration pulls) before recovering; moderate rises as Flash fills a quality niche previously inaccessible at that cost tier.

5 Related Work

Supervised and static routers. The simplest cost-aware strategy is cascading: FrugalGPT (Chen et al., 2024a) queries models cheapest-first and stops once a quality threshold is met. Much subsequent work trains a single-dispatch classifier offline, differing mainly in the training signal—preference pairs (Ong et al., 2025), contrastive or graph-structured embeddings (Chen et al., 2024b; Shirkavand et al., 2025; Feng et al., 2025), or lightweight binary triage (Ding et al., 2024; Pulishetty et al., 2025). OmniRouter (Mei et al., 2025) goes further by formulating budget-aware routing as a batch-level constrained optimization, but still over a fixed prompt set. These methods perform well when deployment matches training, yet leave three structural opportunities open: promoting cost from a manually tuned weight to a closed-loop constraint, adapting autonomously to price or quality shifts, and incorporating new models without retraining. ParetoBandit replaces the frozen policy with online learning, enabling adaptation without retraining or downtime.

Adaptive and bandit-based routers. A growing line of work frames routing as online learning, with each system targeting a different axis of the problem. On the *cost* axis, PILOT (Panda et al., 2025) pairs LinUCB with an online multi-choice knapsack to respect cumulative spend limits, and PROTEUS (Bhatti et al., 2026) trains a Lagrangian RL policy that minimizes cost subject to an accuracy floor τ. On the *quality and generalization* axis, BaRP (Wang et al., 2025a) uses entropy-regularized policy gradients conditioned on a preference vector, LLM Bandit (Li, 2025) conditions on model-identity features to transfer across unseen models, and xRouter (Qian et al., 2025) fine-tunes a small LM as a cost-aware RL orchestrator. MixLLM (Wang et al., 2025b) further extends the scope by jointly modeling downstream latency and evolving model pools in a dynamic contextual-bandit framework. Despite these advances, two opportunities remain: providing closed-loop budget pacing over an open-ended stream (PILOT assumes a known horizon; BaRP and LLM Bandit optimize preference-conditioned rewards rather than explicit dollar constraints; PROTEUS freezes its dual variables at deployment), and incorporating bounded-memory adaptation with explicit evaluation under non-stationarity. ParetoBandit addresses both gaps using an

online dual variable for rate-constrained budget pacing and geometric forgetting for bounded-memory adaptation.

Constrained and non-stationary bandits. ParetoBandit composes three theoretical streams: Lagrangian relaxation for bandits with knapsacks (BwK) (Badanidiyuru et al., 2013; Agrawal et al., 2016), adapted to a rate-style budget constraint with smoothed dual updates (Balseiro & Gur, 2019); geometric discounting for non-stationary bandits (Garivier & Moulines, 2011; Russac et al., 2019); and warm-start priors for contextual bandits (Oetomo et al., 2023; Pan et al., 2024). Each ingredient is well studied in isolation; our contribution is their joint integration and empirical evaluation under cost drift, quality degradation, and cold-start onboarding.

6 Conclusion

We presented ParetoBandit, an adaptive LLM routing system that integrates cost-aware contextual bandits with online budget pacing and geometric forgetting. Production LLM serving is inherently non-stationary: providers revise pricing, model quality can regress silently, and new endpoints must be absorbed without downtime. To our knowledge, ParetoBandit is the first router evaluated under all three conditions. Across four experiments, the system never exceeds the ceiling by more than $\sim 4\%$ for any binding budget; under cost drift it exploits a price drop and maintains compliance when pricing is restored; under silent quality degradation it detects the regression, redistributes traffic, and holds budget while baselines without closed-loop cost control overshoot by up to $6.9\times$; and cold-start model onboarding proceeds without reconfiguration.

Beyond these empirical results, the Pareto knee-point hyperparameter protocol (Appendix A) provides a principled method for jointly calibrating bandit hyperparameters across two competing production requirements: stationary routing efficiency and rapid adaptation to price and quality shifts. This improves on single-objective tuning, which typically selects zero forgetting. As multi-provider LLM portfolios become the norm, our results suggest that bandit-driven routing with integrated budget pacing and non-stationarity awareness will be essential for reliable, cost-controlled LLM serving in production.

Limitations. Six limitations scope the current contribution, most of which are addressable by extending the existing framework (see Future Work). First, evaluation is fully offline: all experiments use a fixed reward matrix with injected non-stationarity, whereas live deployment would introduce stochastic feedback delays and additional distribution shift. Second, rewards are deterministic LLM-judge scores; the noise and sparsity of human-in-the-loop feedback remain untested. Third, we enforce a per-request rate budget B, so sudden traffic spikes can still exceed an aggregate dollar cap even when per-request compliance holds. Fourth, online model onboarding incurs a bounded exploration cost that is paid on production traffic. Fifth, the current formulation is not latency-aware and does not model downstream inference delay or endpoint congestion. Sixth, ParetoBandit constrains cost while maximizing quality, but some deployments instead require the dual objective of minimizing cost subject to a quality floor.

Future work. *Live deployment* (i, ii) is the most immediate next step. This requires computing prompt embeddings online rather than from a cached matrix, and tuning γ for settings where reward labels arrive for only a fraction of requests and with variable delay. Demonstrating that the adaptation and budget-pacing behaviours observed in simulation persist under real traffic would close the largest external-validity gap.

Latency awareness (v) maps naturally onto the BwK framework as a second dual variable, allowing the router to jointly respect cost ceilings and tail-latency SLAs and thus avoid routes that are budget-optimal but violate response-time guarantees. *Aggregate budget enforcement* (iii) could be layered on via a token-bucket mechanism that caps total spend over a billing window, though practitioners who set the per-request ceiling at a profitable operating point may find an additional aggregate cap unnecessary, since higher traffic then increases both spend and revenue. Finally, *quality-constrained routing* (vi) inverts the pacer to track reward against a floor τ, providing an online counterpart to PROTEUS (Bhatti et al., 2026) for deployments where a minimum quality SLA is binding and cost should be minimized subject to it. Validating these extensions under realistic deployment conditions is a natural next step.

References

Acquaviva, C. AI gateway benchmark: Kong AI Gateway, Portkey, and LiteLLM. https://konghq.com/blog/engineering/ai-gateway-benchmark-kong-ai-gateway-portkey-litellm, 2025. Proxy-only benchmark on c5.4xlarge EKS nodes (16 vCPU). Kong p95 = 24 ms, Portkey p95 $\approx$ 70 ms; WireMock baseline p95 = 24 ms.

Agrawal, S. and Goyal, N. Thompson sampling for contextual bandits with linear payoffs. *Proceedings of the 30th International Conference on Machine Learning*, 2013.

Agrawal, S., Devanur, N. R., and Li, L. Linear contextual bandits with knapsacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and

Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Badanidiyuru, A., Kleinberg, R., and Slivkins, A. Bandits with knapsacks. In *Proceedings of the 54th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 207–216. IEEE, 2013.

Balseiro, S. R. and Gur, Y. Learning in repeated auctions with budgets: Regret minimization and equilibrium. *Management Science*, 65(9):3952–3968, 2019.

Bhatti, A. S., Vaddina, V., and Birru, D. PROTEUS: SLA-aware routing via Lagrangian RL for multi-LLM serving systems. *arXiv preprint arXiv:2601.19402*, 2026.

Chen, L., Zaharia, M., and Zou, J. How is ChatGPT's behavior changing over time? *arXiv preprint arXiv:2307.09009*, 2023.

Chen, L., Zaharia, M., and Zou, J. Frugalgpt: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research (TMLR)*, 2024a. URL https://openreview.net/forum?id=cSimKw5p6R.

Chen, S., Jiang, W., Lin, B., Kwok, J. T., and Zhang, Y. Routerdc: Query-based router by dual contrastive learning for assembling large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Ruhle, V., Levi, L. V., and Awadallah, A. H. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations (ICLR)*, 2024.

Feng, T., Shen, Y., and You, J. Graphrouter: A graph-based router for LLM selections. In *International Conference on Learning Representations (ICLR)*, 2025.

Ganglani, K. LLM API latency benchmarks [2026]: 5 models compared. https://kunalganglani.com/blog/llm-api-latency-benchmarks-2026, March 2026. Independent benchmark of five production LLM APIs (GPT-4.1, GPT-4.1 Mini, Gemini 2.5 Flash, Claude Haiku 4.5, Claude Sonnet 4). Three prompt sizes, streaming TTFT, total latency, and throughput.

Garivier, A. and Moulines, E. On upper-confidence bound policies for switching bandit problems. In *Proceedings of the 22nd International Conference on Algorithmic Learning Theory (ALT)*, pp. 174–188. Springer, 2011.

Hedges, L. V. and Olkin, I. *Statistical Methods for Meta-Analysis*. Academic Press, Orlando, FL, 1985.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Kang, Y., Hsieh, C.-J., and Lee, T. C. M. Online continuous hyperparameter optimization for generalized linear contextual bandits. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2302.09440.

Lattimore, T. and Szepesvári, C. *Bandit Algorithms*. Cambridge University Press, 2020.

Li, L., Chu, W., Langford, J., and Schapire, R. E. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pp. 661–670, 2010.

Li, Y. LLM bandit: Cost-efficient LLM generation via preference-conditioned dynamic routing. *arXiv preprint arXiv:2502.02743*, 2025.

Lin, S., Hilton, J., and Evans, O. TruthfulQA: Measuring how models mimic human falsehoods. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.

Liu, X., He, B., Liu, X., Luo, A., Zhang, H., and Chen, H. 98× faster LLM routing without a dedicated GPU: Flash attention, prompt compression, and near-streaming for the vLLM semantic router. *arXiv preprint arXiv:2603.12646*, 2026.

Ma, W., Yang, C., and Kästner, C. (why) is my prompt getting worse? rethinking regression testing for evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering—Software Engineering for AI (CAIN)*, 2024.

Mei, K., Xu, W., Lin, S., and Zhang, Y. OmniRouter: Budget and performance controllable multi-LLM routing. *arXiv preprint arXiv:2502.20576*, 2025.

Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018.

Oetomo, B., Perera, R. M., Borovica-Gajic, R., and Rubinstein, B. I. P. Cutting to the chase with warm-start contextual bandits. *Knowledge and Information Systems*, 65(9):3533–3565, 2023.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. Routellm: Learning to route llms with preference data. In *International Conference on Learning Representations (ICLR)*, 2025.

OpenAI. Prompt caching in the API. https://openai.com/index/api-prompt-caching, October 2024. Pricing table lists gpt-4o-2024-08-06 at $2.50 per million input tokens, down from $5.00 (50% cut); output at $10.00, down from $15.00 (33% cut).

Pan, H., Tennenholtz, G., Mannor, S., Chi, C.-W., Brekelmans, R., Shah, P., and Tewari, A. Jump starting bandits with LLM-generated prior knowledge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 19858–19873, 2024.

Panda, P., Magazine, R., Devaguptapu, C., Takemori, S., and Sharma, V. Adaptive LLM routing under budget constraints. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. arXiv:2508.21141.

Portkey AI. The most reliable AI gateway for production systems. https://portkey.ai/blog/the-most-reliable-ai-gateway-for-production-systems, 2025. Sub-millisecond hot-path latency; 122 KB gateway footprint; >10B requests/month in production.

Pulishetty, R., Ghantasala, M. K., Dasoju, K. K., Mangwani, N., Garimella, V., Mate, A., Chatterjee, S., Kang, Y., Nosakhare, E., Hasan, S., and Srinivasan, S. One head, many models: Cross-attention routing for cost-aware LLM selection. *arXiv preprint arXiv:2509.09782*, 2025.

Qian, C., Liu, Z., Kokane, S., Prabhakar, A., Qiu, J., Chen, H., Liu, Z., Ji, H., Yao, W., Heinecke, S., Savarese, S., Xiong, C., and Wang, H. xrouter: Training cost-aware LLMs orchestration system via reinforcement learning. *arXiv preprint arXiv:2510.08439*, 2025.

Reimers, N. and Gurevych, I. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.

Russac, Y., Vernade, C., and Cappé, O. Weighted linear bandits for non-stationary environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Sakaguchi, K., Le Bras, R., Bhagavatula, C., and Choi, Y. WinoGrande: An adversarial Winograd schema challenge at scale. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

Shirkavand, R., Gao, S., Yu, P., and Huang, H. Cost-aware contrastive routing for LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Sun, Z., Xue, H., Wang, J., and Tu, Z.-X. Lookahead routing for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q. V., Chi, E. H., Zhou, D., and Wei, J. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. *Findings of the Association for Computational Linguistics (ACL)*, 2023.

Verga, P., Hofstätter, S., Althammer, S., Su, Y., Gupta, A., Faez, H., Petroni, F., and Riedel, S. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *arXiv preprint arXiv:2404.18796*, 2024.

Wang, W., Yang, T., Chen, H., Zhao, Y., Dernoncourt, F., Rossi, R. A., and Eldardiry, H. Learning to route LLMs from bandit feedback: One policy, many trade-offs. *arXiv preprint arXiv:2510.07429*, 2025a.

Wang, X., Liu, Y., Cheng, W., Zhao, X., Chen, Z., Yu, W., Fu, Y., and Chen, H. MixLLM: Dynamic routing in mixed large language models. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 10912–10922, 2025b.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a machine really finish your sentence? *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

Zhao, J., Shin, C., Huang, T.-H., Namburi, S. S. S., and Sala, F. CARE: Confounder-aware aggregation for reliable LLM evaluation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

A Hyperparameter Optimization for Non-Stationary Routing

The challenge. ParetoBandit's core hyperparameters—exploration coefficient α, prior strength n_{eff}, and forgetting factor γ—interact non-trivially with non-stationarity. Standard cross-validation optimizes a single stationary metric, which tends to select $\gamma \geq 0.999$ because forgetting reduces effective sample size under stationarity. However, γ=1.0 fails under non-stationarity (Section 4.4): infinite memory retains stale pre-failure evidence, slowing adaptation to quality degradation, while optimizing solely for adaptability yields excessive forgetting that destabilizes the stationary policy. A good configuration must balance stationary efficiency against non-stationary recovery.

Among bandit-based LLM routers we surveyed, none addresses this tension directly. PILOT (Panda et al., 2025) tunes α via grid search on a single stationary metric. BaRP (Wang et al., 2025a) and LLM Bandit (Li, 2025) omit forgetting entirely. MixLLM (Wang et al., 2025b) delegates adaptation to periodic neural retraining rather than explicit forgetting. In the general bandit literature, CDT (Kang et al., 2024) performs online tuning of α via a bandit-over-bandit scheme but remains single-objective and does not tune γ. Garivier and Moulines (Garivier & Moulines, 2011) derive γ as a function of the (unknown) number of breakpoints, which is not observable in production.

Adaptation–horizon coupling. Rather than tuning n_{eff} and γ as independent statistical parameters, we reparameterize both through a single deployment-meaningful quantity: the adaptation horizon T_{adapt}—the number of online queries after which the router can fully override its prior following a distributional shift. Under the Discounted LinUCB update (Russac et al., 2019), the effective weight of online evidence relative to the prior reaches parity after

$$T_{\text{adapt}} = -\frac{\log\big(n_{\text{eff}}\,(1-\gamma)+1\big)}{\log\gamma} \tag{13}$$

queries. Inverting for n_{eff} gives $n_{\text{eff}} = (\gamma^{-T_{\text{adapt}}}-1)/(1-\gamma)$, which reduces to $n_{\text{eff}} = T_{\text{adapt}}$ as $\gamma \to 1$ by L'Hôpital's rule. Fixing T_{adapt} collapses the 3D grid $(\alpha, n_{\text{eff}}, \gamma)$ to a 2D search over (α, γ) with n_{eff} derived. The practitioner sets one number they understand—the shortest acceptable reaction time for their service (e.g., the time to detect and reallocate away from a degraded model)—rather than two abstract statistical parameters whose interaction with non-stationarity is unintuitive. Every configuration in the resulting sweep adapts at the same rate; only the exploration–exploitation balance (α) and the granularity of forgetting (γ) differ.

We anchor $T_{\text{adapt}} = 500$ to the catastrophic-failure phase length ($N_{\text{phase2}} \approx 595$ prompts in the validation split), ensuring the router can fully override its prior within the experimental measurement window.

Multi-objective formulation. With n_{eff} derived, we score each (α, γ) configuration on two objectives:

1. **Budget-paced Pareto AUC** (stationary efficiency). On the validation split, we sweep log-spaced budget targets with the BudgetPacer active, build per-seed Pareto frontiers, and compute the area under each frontier. We then average AUC over $6 \times 7 = 42$ configurations and 20 random seeds.
2. **Catastrophic-failure Phase-2 reward** (non-stationary resilience). We run a two-phase simulation on the validation split: Phase 1 (first half) uses normal rewards; Phase 2 (second half) degrades the failed arm's reward to 0.50 ($\sim$46% below normal) while keeping its cost fixed. We use a stronger degradation than Experiment 3's 0.75 so that the selected configuration can detect failures of at least this severity; the recovery-limit study (Appendix G) confirms the system handles even larger degradations. Mean Phase-2 reward captures how quickly the router detects the failure and reallocates traffic. We tune on a Mistral failure—the most operationally stressful scenario, since Mistral is the dominant arm across the broadest range of budget targets—and validate generalization to all K arms post hoc.

Pareto knee-point selection. We select a single configuration from the scored grid by constructing the Pareto frontier of non-dominated (AUC, Phase-2 reward) pairs and identifying the knee point. Concretely, we compute each frontier point's perpendicular distance to the line connecting the two extreme endpoints, after min–max normalisation of both objectives for scale invariance, and choose the point with maximal distance. Geometrically, the knee marks the inflection beyond which improving either objective requires a disproportionate sacrifice of the other; unlike scalarisation weights or ε-constraints, it requires no user-specified trade-off parameter. Seed-level bootstrap resampling (2000 iterations) shows this choice is stable: 91.0% of resamples select a configuration within one γ-grid step of the knee (only 6 unique configurations appear), with the knee itself chosen in 50.9% of resamples, the modal selection by a wide margin.

Results. Table 3 compares knee-point selection against AUC-only optimisation for both ParetoBandit (with warmup priors) and Tabula Rasa (cold start). The contrast between the two selection methods is stark. AUC-only optimisation selects γ=1.0 (no forgetting) for both ParetoBandit and Tabula Rasa, converging on stationary optima that Section 4.4 shows are vulnerable under quality degradation. The knee-point method instead selects moderate forgetting for both variants—γ=0.997 for ParetoBandit and γ=0.997 for Tabula Rasa—trading only $\sim$0.08% stationary AUC (for ParetoBandit) for substantially improved failure resilience.

The selected configurations generalise to the held-out

Table 3. T_{adapt}**-constrained Pareto knee-point selection.** The knee-point method selects moderate forgetting (γ=0.997 / 0.997) for both variants, while AUC-only optimisation favours near-stationary γ. Warmup priors yield higher AUC, higher Phase-2 reward, and lower variance than the cold-start baseline. Grid: 6 α × 7 γ values; val split, 20 seeds.

Variant	Method	α	n_{eff}	γ	BP AUC	P2 Reward
ParetoBandit	AUC-only	0.01	500	1.0	0.928	—
	Knee-point	0.01	1164	0.997	0.928	0.7312
Tabula Rasa	AUC-only	0.10	—	1.0	0.925	—
	Knee-point	0.05	—	0.997	0.923	0.7287

test split. On the test split, the router–envelope AUC gap is -0.35% for ParetoBandit and -0.58% for Tabula Rasa, both within one standard deviation of seed variance (± 0.0012 and ± 0.0020, respectively), indicating that the offline protocol does not overfit. The forgetting tax under stationarity is negligible: ParetoBandit reaches 0.9221 [0.9216, 0.9226] on the test split, only $\sim 0.35\%$ below the fixed-model envelope (0.9253), despite using a forgetting rate tuned for non-stationary recovery. Warmup priors close roughly 40% of the remaining gap to the envelope: ParetoBandit at 0.9221 [0.9216, 0.9226] versus 0.9200 [0.9191, 0.9208] for Tabula Rasa.

Cross-arm validation. Because the Phase-2 objective is tuned on Mistral failure alone—the most operationally stressful scenario, as Mistral is the dominant arm across the widest range of budget targets—we verify that the selected forgetting rate generalises by evaluating the knee-point configuration under catastrophic failure of each of the K=3 arms independently (validation split, 20 seeds, 3 budget targets). Phase-2 mean reward is 0.8470 [0.8140, 0.8800] under Llama failure, 0.7312 [0.7231, 0.7393] under Mistral failure (the tuning target), and 0.8916 [0.8852, 0.8980] under Gemini failure (all 95% CIs). Mistral failure remains the hardest case (lowest Phase-2 reward), so tuning on the dominant arm is a conservative choice and the forgetting mechanism generalises across arms.

Three findings stand out from the joint selection:

1. T_{adapt} **reduces the search from 3D to 2D.** Deriving n_{eff} from the adaptation horizon T_{adapt} (Eq. 13) collapses the $(\alpha, n_{\text{eff}}, \gamma)$ grid to (α, γ). The practitioner sets one deployment-meaningful quantity—the shortest acceptable reaction time—instead of two abstract statistical parameters.
2. **Knee-point selection enables forgetting that threshold methods miss.** An ε-best-AUC threshold would select γ=1.0 for Tabula Rasa because every $\gamma < 1$ falls outside the tolerance band. The knee-point criterion instead identifies where the marginal AUC cost of additional forgetting accelerates, and selects γ=0.997 even on the steeper trade-off curve.
3. **Warmup priors and forgetting are synergistic.** ParetoBandit dominates Tabula Rasa on both objectives (0.928 vs. 0.923 AUC; 0.7312 vs. 0.7287 Phase-2 reward). Forgetting ($\gamma < 1$) down-weights all past evidence, including the prior embedded in the sufficient statistics at initialisation. For Tabula Rasa the decayed prior is uninformative $(\lambda I, \mathbf{0})$; for ParetoBandit it retains the structure of the warmup characterisation, providing a better-than-ignorance fallback even after substantial decay. Warmup priors therefore matter more when forgetting is active, because they determine the fallback the router reverts to after a disruption; cross-arm validation confirms that this advantage generalises to all arms.

T_{adapt} **sensitivity.** A natural concern is whether the selected configuration is sensitive to the anchor value T_{adapt}=500, which we set to approximately match the catastrophic-failure phase length. To test robustness, we repeat the full Pareto knee-point selection for $T_{\text{adapt}} \in \{250, 500, 1000\}$—a $4\times$ range spanning reaction times from rapid detection to conservative adaptation (Table 4). Three observations support robustness:

1. α **is perfectly stable** (0.01 at every T_{adapt}), as expected: α is orthogonal to the adaptation horizon.
2. γ **shifts but remains in the moderate-forgetting regime.** The selected values—0.996 (T_{adapt}=250), 0.997 (T_{adapt}=500), and 0.994 (T_{adapt}=1000)—span only three grid steps ($\Delta\gamma = 0.003$), with derived n_{eff} of 431, 1 164, and 68 298, respectively. As T_{adapt} increases, the derived n_{eff} grows and the Pareto criterion compensates with slightly more aggressive forgetting. Crucially, every selected γ remains well below the γ=1.0 regime with no forgetting, so the qualitative behaviour—moderate discounting with non-stationary resilience—is preserved for all anchors.
3. **Both objectives are effectively invariant.** Despite the three-step γ shift, AUC varies by less than 0.25% (0.9255–0.9277) and Phase-2 reward spans only 0.7933–0.7945, with no monotonic trend; the performance surface near the Pareto knee is flat, so T_{adapt} mainly affects the internal parameterisation (γ, n_{eff}) rather than external routing behaviour.

A practitioner can therefore set T_{adapt} directly from an operational requirement—the shortest acceptable reaction time—knowing that the knee-point procedure will adjust γ and n_{eff} automatically while keeping routing performance essentially unchanged.

Table 4. T_{adapt} **sensitivity analysis.** Pareto knee-point selection repeated for $T_{\text{adapt}} \in \{250, 500, 1000\}$, re-deriving n_{eff} from γ at each value. Grid: $\alpha \in \{0.01, \ldots, 1.0\}$ (6 values) $\times$ $\gamma \in \{0.994, \ldots, 1.0\}$ (7 values), val split, 20 seeds.

T_{adapt}	α	γ	n_{eff}	BP AUC ↑	P2 Reward ↑
250	0.01	0.996	431	0.9255	0.7934
500	0.01	0.997	1164	0.9277	0.7933
1000	0.01	0.994	68298	0.9267	0.7945

B Cost Heuristic Validation

The selection utility (Equation 2) uses a static log-normalized cost $\tilde{c}_a$ derived from each model's blended per-token rate rather than the realised per-request cost. This is forced by decision timing: the cost penalty must be evaluated before the prompt is dispatched, but realised cost depends on the model's output length, which is unknown until inference completes. A learned contextual cost predictor could in principle replace the static proxy, but the benefit is limited—prompt-level features explain less than 8% of cost variance in our portfolio (see "Prompt-cost correlation" below)—while the dual variable λ_t already corrects for any systematic mismatch by updating on actual costs (Eq. 4).

We validate the static heuristic via two necessary conditions: (i) $\tilde{c}_a$ preserves the true cost ranking across prompts, and (ii) within-model cost variance is small relative to inter-model gaps in log-cost space. Condition (i) is required because the cost penalty in Eq. 2 biases selection toward cheaper arms; if the heuristic mis-ranks two models, the penalty systematically favours the wrong one. Condition (ii) ensures that a single scalar per arm is a faithful summary of its cost distribution: if within-model variance were large relative to inter-model gaps, the fixed penalty could not reliably separate tiers and prompt-level cost estimation would be needed.

Blending assumption. The heuristic blends input and output pricing with equal weight (i.e., a 1:1 token ratio). In practice the output-to-input ratio varies across models and prompts, so the blended rate differs from the true per-request effective rate. Because cost ranking depends only on the relative order of effective rates and log-normalization compresses the dynamic range, a simple average preserves the ranking whenever the price gap between models exceeds the variation introduced by differing output lengths—a condition satisfied for models spanning at least one order of magnitude in pricing. Note that Llama-8B's blended rate ($0.10/M tokens) coincides with the market cost floor, so $\tilde{c}_{\text{Llama}} = 0$ by construction; any model priced at or below the floor is treated as zero-cost in the utility computation.

$K{=}3$ **portfolio (1766 shared validation prompts; sanity check).** The heuristic ordering Llama-8B $<$ Mistral-Large $<$ Gemini-Pro matches the per-request cost ordering on 100.0% of prompts (95% Wilson CI: [99.8, 100.0]%). All three pairwise comparisons hold at 100.0% under strict inequality, with zero ties observed. This result is unsurprising given the $\sim$500$\times$ cost span between the cheapest and most expensive arm (per-model CVs in 0.63–0.92), and serves primarily as a sanity check that the dataset is internally consistent (Figure 6a; Figure 7a).

$K{=}4$ **portfolio (1766 shared validation prompts, with Gemini-Flash).** Adding a fourth arm whose pricing overlaps an existing arm provides a harder test of the heuristic. For a clean comparison, we restrict both portfolios to the shared prompt subset: 19 prompts from the original $K{=}3$ validation split are excluded because the Flash data-collection pipeline failed to produce a successful response or judge score for them, so they are absent from the $K{=}4$ data file. Gemini-Flash ($\tilde{c} = 0.382$) is placed between Mistral-Large ($\tilde{c} = 0.333$) and Gemini-Pro ($\tilde{c} = 0.583$). The full ordering matches on 79.7% (CI: [77.7, 81.5]%) of prompts. The closest pair (Mistral-Large vs. Gemini-Flash) preserves ranking on 79.7% (CI: [77.7, 81.5]%; 0 exact ties), reflecting Flash's high cost variance (CV $= 1.56$) and the narrow heuristic gap ($\Delta\tilde{c} = 0.049$) (Figure 6b; Figure 7b).

Log-cost separation. To quantify tier separation without referencing the heuristic itself, we examine per-model distributions in $\log(\text{cost}_{\text{USD}})$ space. Within-model standard deviations are 8–11% of the total inter-model log-cost range, confirming that prompt-level cost noise does not overwhelm tier separation. We report Cohen's d between adjacent tiers as a scale-free measure of how many within-model standard deviations separate each pair; $d \gg 1$ implies that the two distributions rarely overlap, so a single scalar per arm faithfully distinguishes them (condition (ii) above). In the $K{=}3$ portfolio, Cohen's d for adjacent pairs ranges from 4.39 (Llama-8B $\rightarrow$ Mistral-Large) to 5.92 (Mistral-Large $\rightarrow$ Gemini-Pro)—well above $d{=}1$, consistent with the 100% ranking preservation reported above. In the $K{=}4$ portfolio, the Mistral-Large $\rightarrow$ Gemini-Flash pair has $d = 0.68$; this weakest separation corresponds to the only pair where the ranking inverts on $\sim$20% of prompts.

Prompt-cost correlation. Per-model cost distributions are right-skewed (Figure 6), so we use Spearman (rank-based) correlations between prompt word count and per-request cost. Correlations are statistically significant but modest in magnitude ($\rho = 0.12$ to 0.27; all $p < 10^{-4}$). Because $\rho^2 < 8\%$ (proportion of rank variance explained), prompt-level features capture only a small fraction of cost variation, limiting the practical benefit of a contextual cost model in this portfolio.

Cross-model cost correlation. If per-request costs covary across models (e.g. because a long prompt elicits long outputs from every model), then prompt-level cost fluctu-

ations shift all arms together, preserving relative ordering even when absolute costs vary. We again use Spearman correlations given the skewed cost distributions. Per-request costs are moderately to strongly correlated across models ($\rho = 0.56$ to 0.68), confirming this shared output-length factor and explaining why the static heuristic's ranking holds despite high within-model CVs (0.63–1.56).

Takeaway. The static $\tilde{c}_a$ heuristic is empirically well-calibrated for portfolios where models span at least one order of magnitude in pricing. For denser price clusters (e.g., Mistral-Large vs. Gemini-Flash, $d = 0.68$), the ranking holds for $\sim$79.7% of prompts but inverts when Flash generates unusually short responses. Three structural properties limit downstream impact: (i) $\tilde{c}_a$ enters only the soft penalty (Eq. 2); the hard ceiling and the pacer's EMA cost signal operate on actual per-request costs, so heuristic errors cannot cause budget violations. (ii) For closely priced arms the absolute penalty gap is small ($\sim$0.015 for Mistral–Flash at λ_t=0)—negligible relative to the $[0, 1]$ reward signal. (iii) The dual variable λ_t updates on actual costs (Eq. 4), self-correcting any persistent over-selection of a costly arm.

C Cold-Start vs Warmup Priors

Section 3.4 introduces offline-to-online warmup priors, which let the router begin with informed beliefs about each model's strengths by loading sufficient statistics fitted on historical data. The natural question is whether warmup priors reduce cold-start regret in practice. To answer this, we compare ParetoBandit with warmup priors (α=0.01, n_{eff}=1164, γ=0.997) against Tabula Rasa (α=0.05, n_{eff}=1, γ=0.997), each running under its own Pareto-optimal hyperparameters tuned independently via the Pareto-knee procedure (Appendix A). Notably, the sweep independently selected the same γ=0.997 for both conditions, so they share identical effective-memory windows ($\sim$333 steps) and the only remaining differences are prior strength (n_{eff}=1164 vs. 1) and exploration rate (α=0.01 vs. 0.05). Any regret difference is therefore directly attributable to the informative priors, not to a memory-length confound.

The comparison uses the K=3 stationary portfolio under four budget regimes: unconstrained (no pacer), tight (B=3×10^{-4}/request), moderate (B=6.6×10^{-4}/request), and loose (B=1.9×10^{-3}/request).

Statistical methodology. We assess each comparison along two dimensions. The first is typical-case regret: does warmup tend to outperform seed by seed? The second is tail risk: does warmup avoid outlier runs where regret is disproportionately high? We call a seed a catastrophic failure when its cumulative regret exceeds 2× the pooled median across all conditions in that budget regime; pooling ensures no single condition anchors the threshold. These two dimensions require different tests because a method can improve the median while still leaving heavy tails, or eliminate outliers without shifting the centre.

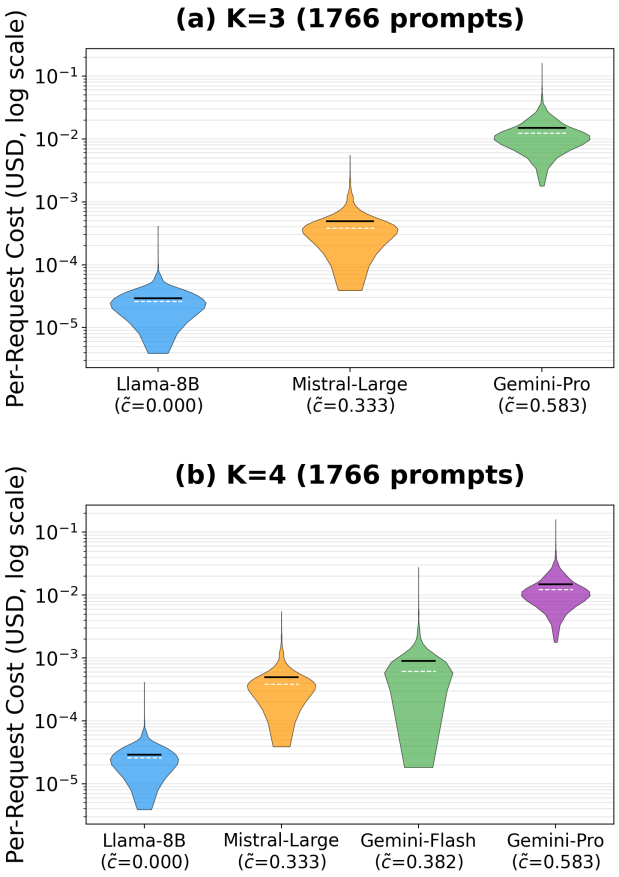


Figure 6. Cost heuristic validation: realised cost distributions. Panel (a) shows per-request realised cost distributions for the K=3 portfolio on the shared validation subset, and panel (b) shows the same diagnostic for the K=4 portfolio. In both cases the violin plots are ordered by the static log-normalized heuristic $\tilde{c}_a$, illustrating that the major cost tiers are cleanly separated, with only the Mistral-Large and Gemini-Flash distributions showing substantial overlap.

- **Sign test** (exact binomial): tests whether warmup has lower regret seed by seed, capturing the typical case (H_0: $P(\text{warmup wins}) = 0.5$).
- **Fisher exact test** on a 2×2 table of catastrophic vs. non-catastrophic seeds: tests whether warmup has a lower catastrophic-failure rate, capturing tail risk (H_0: equal failure rates).

All p-values are corrected for multiplicity via the Holm–Bonferroni procedure across the full family of tests (4 sign tests and 4 Fisher tests, corrected separately per family). Bootstrap CIs for paired differences are likewise Bonferroni-corrected (4 simultaneous comparisons) to maintain 95% family-wise coverage.

Table 5. Warmup-prior ablation results across budget regimes (20 seeds, held-out test split, n=1824). Warmup priors are compared against Tabula Rasa and a random baseline (unconstrained only). Regret is cumulative over all 1824 test steps; R@200 is cumulative regret at test step 200. CIs are 95% percentile-bootstrap (10,000 resamples). *Holm–Bonferroni corrected. **Cat.** = catastrophic failures (regret $> 2\times$ pooled median of all conditions in each budget regime).

Budget	Condition	**Regret** (95% CI)	Std	**R@200** (95% CI)	Rwd	Cat.	p^*_{sign}	p^*_{Fisher}
None	Warmup	**55.0** [54.3, 55.7]	1.6	**6.2** [5.8, 6.6]	**0.933**	0/20	—	—
	Tabula Rasa	87.0 [77.1, 99.0]	24.8	16.2 [11.9, 20.8]	0.916	2/20	$<10^{-5}$	0.974
	Random	146.5 [144.4, 148.8]	5.0	16.0 [15.1, 17.0]	0.883	—	—	—
Tight	Warmup	**161.2** [159.1, 163.1]	4.6	**19.5** [18.0, 21.0]	**0.875**	0/20	—	—
	Tabula Rasa	213.3 [195.8, 232.1]	41.8	29.2 [26.4, 31.9]	0.846	0/20	0.004	1.000
Moderate	Warmup	**133.9** [129.2, 138.8]	11.0	**14.7** [12.9, 16.6]	**0.890**	0/20	—	—
	Tabula Rasa	147.5 [130.0, 165.3]	40.5	23.5 [19.4, 27.5]	0.882	0/20	0.412	1.000
Loose	Warmup	**83.6** [81.4, 85.9]	5.1	**8.6** [8.2, 9.1]	**0.918**	0/20	—	—
	Tabula Rasa	114.0 [101.1, 128.3]	30.8	22.2 [18.3, 26.0]	0.901	0/20	0.004	1.000

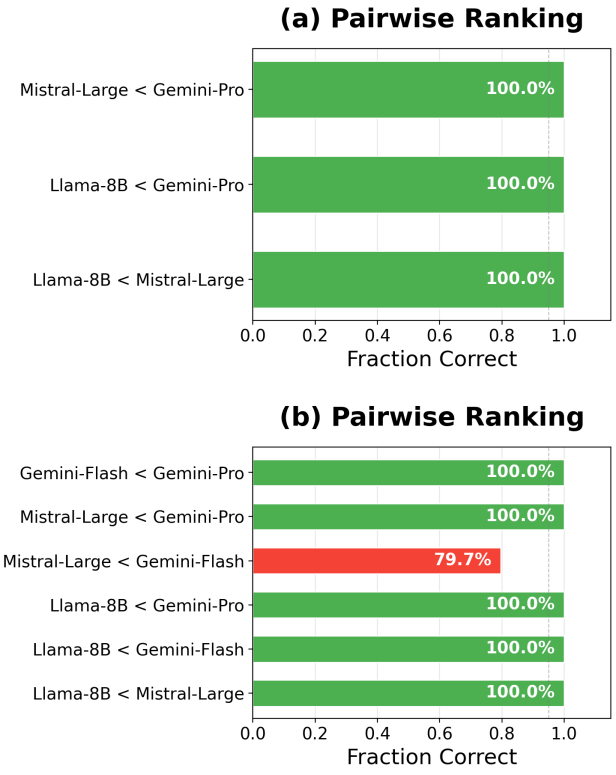


Figure 7. Cost heuristic validation: ranking preservation. Panel (a) shows pairwise ranking preservation for the K=3 portfolio on the shared validation subset, and panel (b) shows the same diagnostic for the K=4 portfolio. The heuristic preserves the full per-request cost ordering on 100.0% of shared-subset K=3 prompts and ∼79.7% of shared-subset K=4 prompts; the Mistral-Large vs. Gemini-Flash comparison is the only source of inversions.

Table 5 reports per-condition means, bootstrap CIs, standard deviations, early-learning regret (R@200), mean reward, catastrophic-failure counts, and Holm-corrected p-values; below we highlight the main findings.

Result: warmup priors reduce cold-start regret. Relative to Tabula Rasa, warmup priors reduce mean cumulative regret across all budget regimes. We report the paired difference Δ = Tabula Rasa − Warmup (positive favours warmup) with Bonferroni-corrected 95% bootstrap CIs (4 simultaneous comparisons): unconstrained Δ=32.0 [19.8, 48.4], tight Δ=52.2 [29.6, 76.9], moderate Δ=13.7 [−10.9, 38.5], and loose Δ=30.4 [13.9, 49.2]. The CI excludes zero for unconstrained, tight, and loose regimes; the moderate regime is inconclusive (Δ=13.7, CI spans zero). These reductions correspond to 9–37% of Tabula Rasa's total regret (largest in the unconstrained regime, smallest in the moderate regime where the CI spans zero), reflecting that the cold-start baseline converges within the ∼333-step effective-memory window afforded by γ=0.997. Much of this advantage is concentrated in the early learning phase. The R@200 paired differences (Δ = TR − Warmup, same sign convention) are significant in every regime: unconstrained Δ=10.0 [4.7, 15.8], tight Δ=9.7 [5.5, 13.7], moderate Δ=8.8 [3.6, 13.8], and loose Δ=13.6 [8.2, 18.8] (see also per-condition R@200 CIs in Table 5). The advantage narrows as online evidence accumulates beyond the effective-memory window.

Cold-start variability. Warmup priors produce tighter per-seed regret distributions (Figure 8). Under tight budgets, the per-seed regret standard deviation for warmup is 4.6 compared to 41.8 for Tabula Rasa (∼9× lower). Warmup records zero catastrophic failures (regret $> 2\times$ pooled median) across all regimes. Tabula Rasa produces 2/20 catastrophic failures in the unconstrained regime, with none under budget constraints (0/20 tight, 0/20 moderate, 0/20

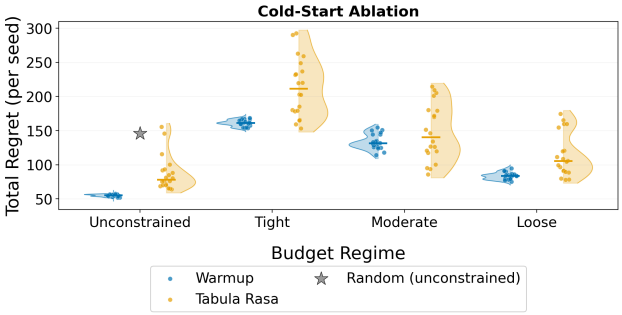


Figure 8. Per-seed cumulative regret distributions for warmup priors (blue) vs. Tabula Rasa (orange) across four budget regimes (K=3, 20 seeds, held-out test split). Both conditions share γ=0.997 but differ in prior strength and exploration rate (deployment-level comparison). Gray star: Random baseline median (unconstrained only). Warmup priors yield tight, unimodal distributions; Tabula Rasa exhibits heavier tails under budget constraints. Statistical tests in Table 5.

loose); Fisher tests are non-significant after Holm correction ($p^*_{\text{Fisher}} = 0.974$). The occasional cold-start outliers confirm that starting without priors introduces tail risk, even when the effective-memory window is the same (γ=0.997 for both conditions).

Discussion. Warmup priors are best understood as a deployment-reliability mechanism: they reduce cold-start regret and tighten the per-seed distribution. Warmup is optional—ParetoBandit is fully functional without it, and the Tabula Rasa results in Table 5 confirm that the bandit converges to comparable steady-state quality once sufficient online evidence accumulates. The warmup benefit is transient by design: geometric forgetting (γ=0.997) replaces priors within ∼333 effective-memory steps, after which warmup and tabula-rasa conditions converge. Enabling warmup priors reduces regret during this early learning window and tightens run-to-run variability, but practitioners who lack historical data or prefer a simpler deployment can omit them without sacrificing long-run performance. When warmup priors are used, these gains assume the prior is directionally correct; a dedicated prior-mismatch sensitivity analysis (Appendix D) quantifies trade-offs across five quality levels—from well-calibrated through domain-restricted to actively inverted priors.

D Prior Mismatch Sensitivity

The warmup ablation (Appendix C) demonstrated that well-calibrated priors substantially reduce regret and eliminate catastrophic cold-start failures under stationary conditions. All experiments in this paper use priors trained on a representative training set drawn from the same distribution as the test traffic. In practice, however, prior quality degrades after deployment: the prompt distribution shifts as users adopt new workflows, model providers silently update weights, and the roster itself changes as models are added or retired. A practitioner deploying warmup priors needs to know how wrong the prior can be before cold start becomes the safer option, and whether a conservative prior strength can limit the damage when calibration quality is uncertain.

Why not re-optimise? The production hyperparameters (Appendix A) are tuned once on a representative training set via the T_{adapt}-constrained Pareto-knee procedure, which couples n_{eff} and γ through the adaptation horizon. The selected prior strength ($n_{\mathit{eff}} \approx 1{,}164$, Eq. 10) implicitly assumes the prior is directionally correct: high n_{eff} tells the bandit to trust the offline beliefs for many observation-equivalents before online evidence takes over. If the prior is wrong, high n_{eff} amplifies the harm by delaying the point at which online data overrides the prior. Re-running the Pareto-knee sweep for every possible prior-quality level is impractical in production, so this experiment instead asks whether a single tunable parameter—n_{eff}—can serve as a safety knob that limits damage from degraded priors while preserving the variance-reduction benefit for good ones.

Design: mismatch gradient. We construct five prior-quality levels forming a mismatch gradient, ordered from ideal to adversarial:

1. **Well-calibrated:** priors from the full training set (n=8,373)—the shipped default.
2. **Random-1680:** a random subsample of 1,680 training prompts, matching the GSM8K-only count while preserving the full distribution. This is a sample-size control: any gap between Random-1680 and Well-calibrated isolates covariance-estimation noise from domain effects.
3. **MMLU-only:** priors from the 1,855 MMLU prompts only. The prior preserves the correct model ranking (Gemini $>$ Mistral $\gg$ Llama) but with knowledge-domain-specific magnitudes.
4. **GSM8K-only:** priors from the 1,680 GSM8K (math) prompts only. All models score ∼0.86+ on math, so the prior encodes near-zero arm differentiation.
5. **Inverted:** priors from the full training set with Llama and Gemini rewards swapped, so the prior believes the cheapest model is best and vice versa.

These levels represent failure modes that arise naturally in deployment: (1–2) an operator who collected representative data; (3) an operator who collected data from a single knowledge domain; (4) an operator whose data happens to come from a task where all models perform similarly; and (5) the worst case, where an update inverts the relative model ranking (e.g., because a provider silently degrades a model that was previously the best).

Design: n_{eff} as a safety knob. Each quality level is tested at three n_{eff} values (10, 100, 1000), spanning three orders of magnitude. These values bracket a key trade-off. Low n_{eff} (≈10) treats the prior as a gentle nudge: the bandit overrides

it within tens of observations, limiting the damage from a bad prior but also limiting the cold-start benefit from a good one. High n_{eff} ($\approx$1,000) trusts the prior for hundreds of effective observations: beneficial when the prior is correct, but potentially harmful when it confidently encodes the wrong ranking. Together with the quality gradient, the 5×3 grid reveals whether there exists an n_{eff} regime where any prior—even a domain-mismatched one—helps more than it hurts.

Design: baseline. The no-prior baseline is the independently optimised Tabula Rasa (α=0.05, γ=0.997). As established in the warmup ablation (Appendix C), the Pareto-knee sweep selected the same γ=0.997 for both warmup and Tabula Rasa, so both share the same effective memory ($\sim$333 steps) and the comparison is not driven by different forgetting horizons. However, because α also differs between conditions (α=0.01 for warmup vs. 0.05 for Tabula Rasa), the appendix should be interpreted as a system-level deployment comparison against the best no-prior baseline rather than a fully controlled ablation that isolates prior information while holding every learning hyperparameter fixed. This yields $5 \times 3 + 1 = 16$ conditions. The warmup hyperparameters were not re-optimised per prior-quality level—matching the production scenario where prior quality degrades after deployment without hyperparameter adjustment.

Design: unconstrained regime. Evaluation follows the same cumulative-regret protocol as the warmup ablation: held-out test split (n=1824), 20 seeds (offset=9,000, aligned for paired comparisons), unconstrained (no budget pacer). We deliberately restrict this experiment to the unconstrained regime for two reasons. First, the budget pacer constrains arm selection, which limits the online observations the bandit receives to correct a bad prior; budget-induced failures and prior-induced failures would be confounded. Second, the warmup ablation (Appendix C) already characterises the pacer interaction for well-calibrated priors vs. tabula rasa. Adding budget constraints here would conflate two failure modes without additional insight, since the prior-quality threshold for harm is determined by the bandit's learning dynamics, not the pacer.

Statistical methodology. We use the same paired-comparison protocol as Appendix C: exact binomial sign test for location shift and Fisher exact test on a 2×2 catastrophic-failure table for tail risk, with Holm–Bonferroni correction across all 15 pairwise comparisons per baseline. All confidence intervals are 95% percentile-bootstrap (10,000 resamples, seed-level resampling); median CIs resample the median directly rather than relying on normality, which is inappropriate for the heavy-tailed baseline distributions observed here. A seed is classified as catastrophic if its regret exceeds $2\times$ the Tabula Rasa median.

Results. Figure 9 summarises total regret across the quality–strength grid.

Directionally correct priors help at every strength. Well-calibrated priors reduce median total regret monotonically with n_{eff}, relative to the Tabula Rasa baseline (median 78.2 [70.7, 90.1]):

- n_{eff}=10: median 65.2 [63.6, 67.3] (16.6% [6.5, 28.5] reduction).
- n_{eff}=100: median 61.9 [60.4, 63.5] (20.9% [11.9, 32.1] reduction).
- n_{eff}=1,000: median 55.5 [54.5, 56.2] (29.1% [21.7, 38.8] reduction).

All well-calibrated and Random-1680 comparisons are significant after Holm correction at every n_{eff} ($p \leq 0.0103$ even at n_{eff}=10), with the strongest effect at n_{eff}=1,000: Well-calibrated wins 20 of 20 seeds ($p < 10^{-4}$); Random-1680 wins 20 of 20 ($p < 10^{-4}$).

Sample size does not matter—distributional match does. A practitioner might worry that the Well-calibrated prior benefits from a larger training set rather than better calibration. The Random-1680 control rules this out: it uses only 1,680 prompts (matching the GSM8K-only count) drawn from the full distribution and performs comparably to Well-calibrated at every n_{eff} (e.g., median 55.1 [54.4, 55.9] vs. 55.5 [54.5, 56.2] at n_{eff}=1,000). This confirms that covariance estimation saturates quickly—1,680 representative prompts are sufficient for d=26 features—and that the domain-mismatch findings below are not confounded by sample-count effects. The practical implication is that operators need a *representative* training sample, not a large one.

Domain-mismatched priors still beat cold start. Even when the prior is trained on a single domain rather than representative traffic, median regret stays below the Tabula Rasa baseline at every n_{eff}—the prior never hurts. MMLU-only priors reduce median regret to 60.1 [58.1, 60.5] at n_{eff}=100 and 58.3 [57.5, 58.7] at 1,000 (vs. Tabula Rasa 78.2 [70.7, 90.1]), despite being trained on a single knowledge domain. GSM8K-only priors—which encode near-zero arm differentiation—lower median regret to 66.7 [65.7, 68.0] through 60.9 [59.4, 62.9] across n_{eff} values. These reductions are statistically significant after Holm correction (Holm-corrected $p < 0.004$ for all domain-mismatched conditions at every n_{eff}). Crucially, no domain-mismatched prior *increases* median regret at any n_{eff}: partial knowledge is consistently at least as good as no knowledge.

Inverted priors cause harm that scales with n_{eff}. This is the only condition where priors hurt. When the prior confidently encodes the wrong model ranking (cheapest = best), the router initially routes traffic to the worst model and must accumulate online evidence to override the prior. Higher n_{eff} delays that override, so damage grows with prior strength (vs. Tabula Rasa median 78.2 [70.7, 90.1]):

- n_{eff}=10: median 78.2 [76.3, 81.8] (modest; the prior is

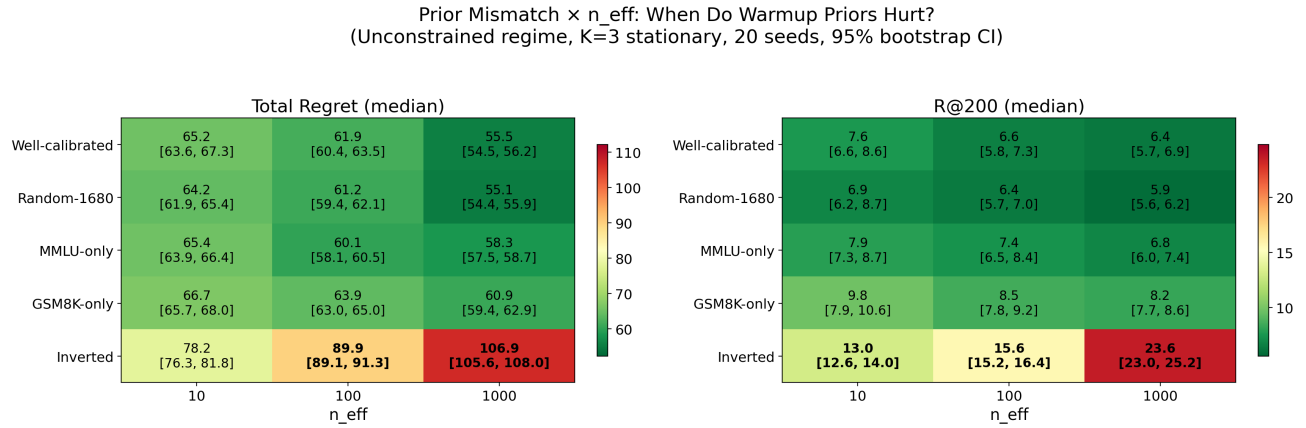


Figure 9. Total cumulative regret for ParetoBandit across the prior-quality vs. prior-strength grid (20 seeds, unconstrained, test split). All conditions use the same ParetoBandit router; only the warmup prior varies. Tabula Rasa is the independently optimised no-prior baseline. **Bold** cells exceed the Tabula Rasa median; normal-weight cells fall below it.

overridden quickly).

- n_{eff}=100: median 89.9 [89.1, 91.3].
- n_{eff}=1,000: median 106.9 [105.6, 108.0]—a 37% increase over the baseline (+29 regret units [16.3, 36.4]).

Tabula Rasa wins 17 of 20 seeds at n_{eff}=1,000 (Holm p=0.0103), confirming that the degradation is both statistically significant and operationally meaningful.

Reliability: the more important finding. For a practitioner, run-to-run consistency matters as much as average regret. A router that usually improves but occasionally catastrophically misroutes is harder to trust than one that reliably delivers moderate gains. Figure 10 shows that warmup priors deliver exactly this consistency: *every* warmup condition—including domain-mismatched and even inverted priors—produces a tight, unimodal per-seed regret distribution, while Tabula Rasa exhibits a wide spread (std=24.8). Concretely, non-inverted warmup conditions have $std \leq 3.2$ (range: 1.6–3.2); even the worst condition (Inverted at n_{eff}=10) achieves std=4.9—still far tighter than Tabula Rasa's 24.8. All 15 warmup conditions record zero catastrophic failures (regret $> 2\times$ the Tabula Rasa median, i.e., > 156.5). The warmup ablation (Appendix C) shows that Tabula Rasa exhibits substantially higher per-seed variance than warmup priors across all budget regimes, with $\sim 9\times$ larger standard deviation under tight budgets; the present experiment extends that finding by showing that the reliability benefit is robust to prior degradation.

Practical implications. The prior-quality threshold for harm is high: priors need to be actively wrong (inverting model rankings) before they degrade performance. The reliability benefit—tight, unimodal distributions vs. Tabula Rasa's heavy tail—is robust across the entire quality gradient. The n_{eff} parameter serves as a safety knob: 1,000 for representative priors (29.1% [21.7, 38.8] regret reduction), 100 as a safe default for domain-shifted priors (20.9% [11.9, 32.1] reduction), and 10 when quality is entirely uncertain (worst-case damage limited to $\sim$0%, while non-adversarial priors still reduce regret by 14.8–17.9%). Tabula rasa is appropriate only when the prior is known to be systematically inverted.

E Reward Signal Robustness (Three-Judge Validation)

All main-paper experiments use DeepSeek-R1 as the sole evaluator. A natural concern is whether the paper's conclusions—the relative ordering of methods and the qualitative behaviour of the router—are artefacts of this particular judge. We address this with a three-judge validation aimed at a single question: would the paper's conclusions change with a different evaluator?

E.1 Setup

We draw a stratified random sample of $n = 2{,}000$ prompts from the 12,000-prompt benchmark, preserving proportional representation of all source benchmarks. Each prompt has existing responses from all three candidate models (Llama-3.1-8B, Mistral-Large-2512, Gemini-2.5-Pro), yielding 6,000 (prompt, model) evaluation pairs. We re-evaluate these fixed responses—no new model outputs—with two supplementary judges chosen for provider diversity: GPT-4.1-mini (OpenAI) and Claude-3.7-Sonnet (Anthropic).

All three judges receive the same structured chain-of-thought evaluation prompt (Figure 11); DeepSeek-R1 additionally receives a brevity instruction to limit its CoT verbosity. The composite reward is a weighted average $r = 0.4\, s_{\text{reason}} + 0.3\, s_{\text{instr}} + 0.3\, s_{\text{comm}}$. The judge panel spans three independent providers (DeepSeek, OpenAI, An-

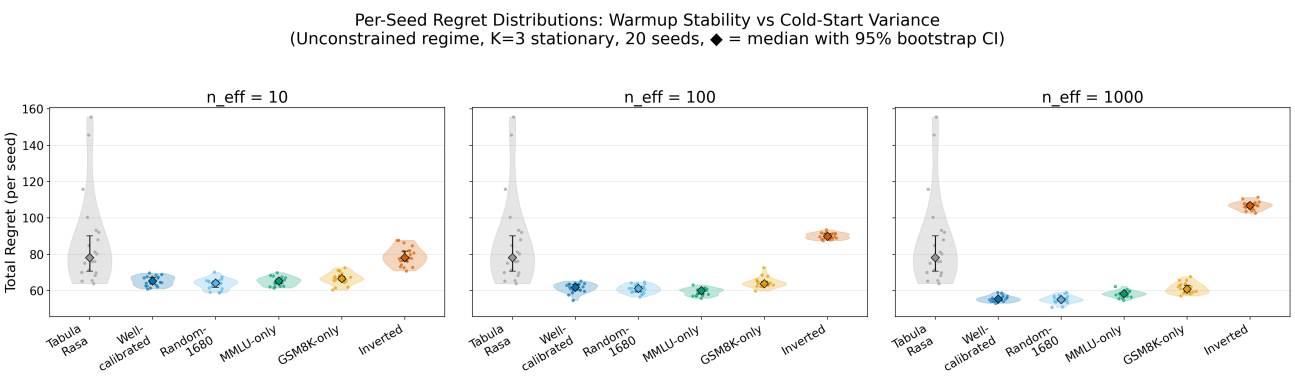


Figure 10. Per-seed regret distributions across prior-quality levels at each n_{eff} value (20 seeds, unconstrained). Tabula Rasa is the independently optimised no-prior baseline.

thropic), none of which overlaps with the routed model providers (Meta, Mistral, Google).

E.2 Population-Level Agreement

The bandit's converged policy depends on the expected reward ordering $\mathbb{E}[r(x, a)]$ across arms, not on individual prompt scores. If this ordering is preserved across judges, the converged policy is invariant to evaluator choice.

Table 6. Expected reward ordering under each judge ($n = 2{,}000$ prompts; Bonferroni-corrected bootstrap CIs, 9 comparisons, $B = 10{,}000$).

Judge	Gemini-Pro	Mistral-Large	Llama-8B
DeepSeek-R1	0.935	0.923	0.798
GPT-4.1-mini	0.964	0.952	0.857
Claude-3.7-Sonnet	0.928	0.915	0.779

Table 6 shows that all three judges rank the models identically: Gemini-Pro $>$ Mistral-Large $>$ Llama-8B. Every pairwise expected-reward gap has the same sign, with all nine Bonferroni-corrected bootstrap CIs excluding zero (narrowest lower bound 0.0046; 9 comparisons, $B = 10{,}000$). Gap-conditioned Kendall's W (Table 9) reaches 0.71 on prompts with large inter-model gaps, indicating that the conditional ordering $\mathbb{E}[r(x, a) \mid x]$ is also largely preserved where routing decisions matter most.

Because all main-paper comparisons are within-judge (same R1 rewards, same R1 oracle), judge choice can change the magnitude of regret differences but not their sign. R1's oracle lift (0.031 per step) is the largest among the three judges (0.016–0.020 for the supplementary judges), so main-paper effect sizes correspond to the most favourable case for adaptive routing; all qualitative conclusions hold under every judge tested.

Cross-judge routing regret. Even where per-prompt routing decisions differ, the practical cost is small. Following R1's oracle policy and evaluating with the other judges' scores yields $\geq 97.4\%$ of their own oracle reward (Table 7). The reverse is worse: following GPT-4.1-mini's or Claude's oracle and evaluating with R1 captures only 95.8% of R1's oracle reward. The asymmetry is partly mechanical: judges with smaller inter-model gaps inflate capture fractions for any routing policy, while R1's larger gaps penalise misrouting more heavily.

Table 7. Cross-judge routing evaluation. Each cell reports the mean reward achieved by following the row judge's oracle, evaluated by the column judge's scores. Parenthetical values show the fraction of the column judge's own oracle reward captured ($B = 10{,}000$ bootstrap resamples).

Train ↓ / Eval →	R1	GPT-4.1-mini	Claude-3.7
R1	0.966 (100%)	0.955 (**97.5%**)	0.923 (**97.4%**)
GPT-4.1-mini	0.926 (95.8%)	0.979 (100%)	0.915 (96.6%)
Claude-3.7-Sonnet	0.926 (95.8%)	0.953 (97.3%)	0.948 (100%)

Panel averaging (Zheng et al., 2023) compresses routing margins: averaging the three judges shrinks the mean inter-model gap from 0.0485 (R1 alone) to 0.0338 (30% compression), halving the fraction of actionable prompts (31% vs. 17% at a 0.05 threshold). Z-score calibration (Hedges & Olkin, 1985) and multi-judge aggregation methods such as PoLL (Verga et al., 2024) and CARE (Zhao et al., 2025) recover < 3 percentage points of this compression, indicating genuine inter-judge disagreement rather than pure scale mismatch. R1 alone provides the largest margins and the lowest per-evaluation cost.

E.3 Per-Prompt Agreement

Table 8 reports per-response agreement using distribution-independent metrics robust to ceiling-compressed rewards (most scores in $[0.8, 1.0]$). Spearman's ρ (0.633–0.658) and Kendall's τ_b (0.528–0.547) indicate moderate rank agreement; mean absolute deviation is ≈ 0.075 and Bland–Altman limits of agreement remain within roughly ± 0.3. This level of LLM-as-judge noise is expected, and the key point is that it preserves population-level ordering and rout-

```
SYSTEM:
You are a Discriminative Router Judge. Your goal is to evaluate
how well an LLM response addresses the given prompt.

Score on three continuous dimensions (0.0-1.0). Use the FULL
range; do NOT default to 0 or 1.

1. Reasoning Quality (40%) - How sound is the reasoning?
   0.9-1.0  Flawless; every step correct and clearly justified.
   0.7-0.8  Sound overall; minor inefficiency or a trivial
            error that does not change the conclusion.
   0.5-0.6  Partially correct; approach is reasonable but
            important steps are wrong or missing.
   0.3-0.4  Weak; only fragments of correct logic.
   0.0-0.2  No coherent reasoning, or completely wrong approach.
   If the prompt needs no multi-step reasoning, score factual
   accuracy and depth of explanation.

2. Instruction Following (30%) - Were all explicit and implicit
   constraints satisfied?
   0.9-1.0  Every constraint followed precisely.
   0.7-0.8  All major constraints met; one minor instruction
            partially missed.
   0.5-0.6  Some important instructions missed or only
            partially addressed.
   0.3-0.4  Multiple instructions ignored or misinterpreted.
   0.0-0.2  Response largely ignores the prompt's requirements.

3. Communication Quality (30%) - How clear, well-structured,
   and useful is the response?
   0.9-1.0  Exceptionally clear, well-organized, appropriate
            detail.
   0.7-0.8  Clear and competent; minor improvements possible.
   0.5-0.6  Adequate but noticeably unclear, verbose, or poorly
            organized.
   0.3-0.4  Hard to follow; significant clarity issues.
   0.0-0.2  Unintelligible, unhelpful, or inappropriate tone.

Format your response EXACTLY as follows:

## Reasoning
<Concise chain-of-thought analysis>

## Reasoning Quality
<0.0 to 1.0>

## Instruction Following
<0.0 to 1.0>

## Communication Quality
<0.0 to 1.0>

USER:
PROMPT: {prompt}

RESPONSE: {response}
```

Figure 11. Structured CoT evaluation prompt sent to all three judges. The composite reward is $r = 0.4\, s_{\text{reason}} + 0.3\, s_{\text{instr}} + 0.3\, s_{\text{comm}}$. For DeepSeek-R1 only, the system message appends: "`IMPORTANT: Keep the ## Reasoning section to 3--5 sentences. Be direct---identify errors or confirm correctness, then move to scoring.`" Temperature is set to 0 for all judges.

ing policies.

Table 8. Per-response agreement with DeepSeek-R1 (6,000 scored pairs per judge). All measures are rank-based or distribution-independent to avoid attenuation from ceiling-compressed rewards.

Metric	GPT-4.1-mini	Claude-3.7-Sonnet
Spearman's ρ	**0.658**	0.633
Kendall's τ_b	**0.547**	0.528
MAD	**0.074**	0.075
Mean bias (judge − R1)	$+0.039$	-0.012
95% Bland–Altman LoA	$[-0.23, +0.37]$	$[-0.36, +0.24]$

Gap-conditioned concordance. Per-prompt best-model agreement with R1 is $\sim$50% overall, but disagreements cluster on prompts where all models are nearly tied. Table 9 shows that when R1's inter-model gap is < 0.05 (30% of prompts), Kendall's $W = 0.17$ and best-model agreement is 48–58%; when the gap is ≥ 0.20 (37% of prompts), W rises to 0.57–0.71 and best-model agreement to 50–58%. In the low-gap region, choosing the "wrong" model costs at most 0.05 reward—less than per-response judge noise (MAD $\approx$0.075)—so disagreement is concentrated where it is cheap.

Table 9. Routing agreement conditioned on R1's inter-model gap. Kendall's W measures three-judge concordance on the full model ranking. Conditioning on R1's gap bins when validating R1 can introduce selection bias; we verify in the reproducibility code that the same monotonic trend holds when conditioning on the consensus (panel-median) gap.

			Best-model agr. vs. R1	
R1 gap range	n	**Kendall** W	**GPT-mini**	**Claude**
$[0.00, 0.05)$	603	0.17	57.5%	48.3%
$[0.05, 0.10)$	367	0.29	37.6%	37.9%
$[0.10, 0.20)$	294	0.43	44.9%	39.5%
$[0.20, 0.30)$	184	0.57	53.3%	49.5%
$[0.30, 1.00)$	552	0.71	56.3%	58.0%
Overall	2,000	0.42	51.3%	47.9%

E.4 End-to-End Regret Under Alternative Judges

We now test whether the bandit's learning dynamics—convergence shape, sample complexity, and relative method ordering—are preserved when the reward signal comes from a different judge, by re-running the bandit simulation under GPT-4.1-mini and Claude-3.7-Sonnet.

Protocol. We split the 2,000 judge-robustness prompts into a stratified $\frac{1}{3}$ validation / $\frac{2}{3}$ test partition (672 / 1,328 prompts). The bandit learns online during the validation burn-in (no metrics), then cumulative regret is evaluated on the held-out test split. To avoid any data-leakage concern, we use cold start (Tabula Rasa) only—no warmup priors—so this is the harder test; if the bandit converges from cold start, warm-start robustness follows. We compare Tabula Rasa against a Random baseline (uniform $1/K$) under four budget regimes (unconstrained, tight, moderate, loose), with 20 seeds per condition. All three judges share the same hyperparameters ($\alpha = 0.05$, $\gamma = 0.997$), tuned originally on R1; judge-specific tuning would only improve the supplementary judges' results, so this choice is conservative.

Results. The bandit converges under all three judges. Under unconstrained routing (Figure 12), Tabula Rasa achieves final cumulative regret of 51.0 [47.7, 54.4] vs. 106.2 [104.6, 107.8] for Random under R1 (52% reduction); 32.8 vs. 71.9 under GPT-4.1-mini (54% reduction); and 37.6 vs. 97.6 under Claude (61% reduction). All curves are sublinear with decreasing slope, consistent with progressive learning of the reward structure regardless of evaluator.

Absolute regret scales with inter-model margins: GPT-4.1-mini's Random baseline (71.9) is 32% lower than R1's (106.2), with Claude intermediate at 97.6. Supplementary/R1 regret ratios are stable across budget regimes (GPT: 0.64 [0.60, 0.68] to 0.89 [0.82, 0.97]; Claude: 0.74 [0.70, 0.79] to 0.92 [0.87, 1.00]), confirming proportional compression rather than a structural change in learning dynamics. Tabula Rasa confidence intervals are wider than Random's, reflecting higher run-to-run variance from cold-start exploration, but the paired difference (Random − Tabula Rasa) is significant for all three judges. Under budget constraints, the qualitative pattern is identical: tighter budgets increase regret, and loose-budget regret approaches the unconstrained level.

E.5 Summary

Across three independent judges, the expected reward ordering is identical (Gemini-Pro > Mistral-Large > Llama-8B; Table 6), R1's oracle captures ≥97.4% of other judges' oracle reward (Table 7), and the bandit's learning dynamics replicate under both supplementary judges, with regret reductions of 52%/54%/61% (R1/GPT/Claude) under unconstrained routing (Figure 12). R1 is therefore well-justified as the sole evaluator: it provides the largest inter-model margins and the lowest per-evaluation cost. Correlating LLM-as-judge scores with human preferences remains important future work, but it is orthogonal to the judge-consistency and routing-robustness validated here.

F Routing Latency Microbenchmark

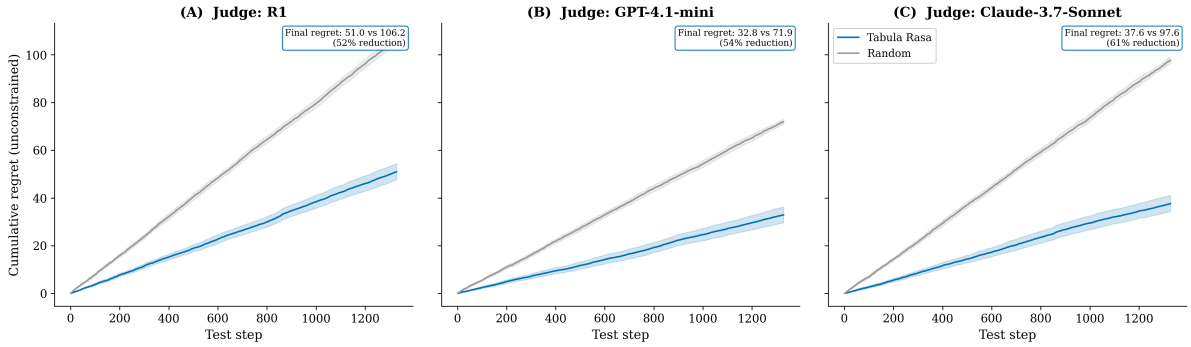
We profile ParetoBandit's per-request routing overhead to validate the computational-efficiency claims in Section 3.5. Eight configurations isolate three factors: Sherman–Morrison (SM) vs. full inversion (at the same abstraction level), production overhead (locks, budget pacing, forgetting), and PCA dimensionality (d=26 vs. d=385). All configurations use K=3 arms, synthetic whitened context vectors, and 4,500 measured route + update cycles (500-round warmup excluded). Contexts are drawn i.i.d. from a standard normal, ℓ_2-normalised, and augmented with a bias term, matching the dimensionality and unit-norm structure of real PCA-whitened embeddings. Synthetic inputs are appropriate here because `route()` and `update()` cost depends only on d and K, not on semantic content; using synthetic vectors removes dataset I/O, encoder inference, and PCA from the timing loop. The full end-to-end pipeline including embedding and PCA is profiled separately below (Table 11).

Table 10. Per-request routing latency (μs) and throughput (K=3 arms, 4,500 measured cycles). Configuration groups are described in the text.

	Route (μs)		Update (μs)		Throughput
Configuration	p50	p95	p50	p95	(req/s)
Production (full router with locks, pacing, forgetting)					
ParetoBandit (d=26)	22.5	26.5	20.4	23.0	22199
ParetoBandit (d=385)	163.0	234.5	456.9	539.0	1582
Algorithmic isolation (identical `route()`, only `update()` differs)					
Bare SM (d=26)	5.8	6.2	8.1	8.4	70459
Bare SM (d=385)	105.2	188.1	320.0	970.8	1608
Cached Inv. (d=26)	7.2	8.0	18.8	19.3	39687
Cached Inv. (d=385)	138.1	230.0	1679.3	2255.2	507
Worst-case baseline (never caches A^{-1})					
Per-Route Inv. (d=26)	50.8	57.3	4.0	5.0	17203
Per-Route Inv. (d=385)	4569.0	4959.3	132.1	785.8	203

Table 10 reports per-request latency and throughput for all configurations (p50, p95, 4,500 cycles). Bare SM and Cached Inv. share the same `route()` code path, isolating the $O(d^2)$ SM vs. $O(d^3)$ full-inversion effect in `update()` only, while ParetoBandit adds production features; Per-Route Inv. never caches A^{-1} and serves as a worst-case baseline. Figure 13 visualises these comparisons on a log scale.

Algorithmic isolation: SM vs. full inversion. Bare SM is a stripped-down LinUCB that applies only the $O(d^2)$ Sherman–Morrison rank-1 update, with no locks, pacing, or forgetting. Cached Inv. is identical except that `update()` calls $O(d^3)$ `np.linalg.inv` instead of the SM correction; both inherit from the same Python base class and



Cumulative regret (mean ± 95% bootstrap CI, 20 seeds). Budget rows omit Random (unconstrained baseline, same across regimes).

Budget	R1 TR	R1 Rand	GPT TR	GPT Rand	GPT/R1	Claude TR	Claude Rand	Claude/R1
unconstrained	51.0± 3.4	106.2± 1.6	32.8± 3.2	71.9± 1.0	0.64	37.6± 3.4	97.6± 1.3	0.74
Budget	R1 TR		GPT TR		GPT/R1	Claude TR		Claude/R1
tight	127.7± 9.5	–	113.1±10.3	–	0.89	117.0± 7.5	–	0.92
moderate	90.6± 5.6	–	76.7± 9.0	–	0.85	77.0± 5.1	–	0.85
loose	65.5± 2.1	–	45.1± 2.8	–	0.69	52.6± 1.1	–	0.80

Note: Wide Tabula Rasa CIs reflect bimodal cold-start dynamics (see Appendix warmup ablation). Most seeds converge quickly; a minority lock onto a suboptimal arm early and accumulate excess regret before correcting. Despite the wide marginal CIs, the paired difference (Random − Tabula Rasa) is significant for all judges (bootstrap 95% CI excludes zero).

Figure 12. Cold-start bandit regret under three judges (1328 test prompts drawn from the 2000-prompt judge-robustness sample). **(A)** R1: 52% regret reduction. **(B)** GPT-4.1-mini: 54% reduction. **(C)** Claude-3.7-Sonnet: 61% reduction. Shaded bands show 95% bootstrap CIs over 20 seeds. Tabula Rasa bands are wider than the Random baseline's due to higher per-seed variance from cold-start exploration. The paired difference (Random − Tabula Rasa) is significant for all three judges (bootstrap 95% CI excludes zero). The summary table reports Tabula Rasa cumulative regret (mean ± 95% bootstrap CI) across all four budget regimes with the supplementary/R1 ratio.

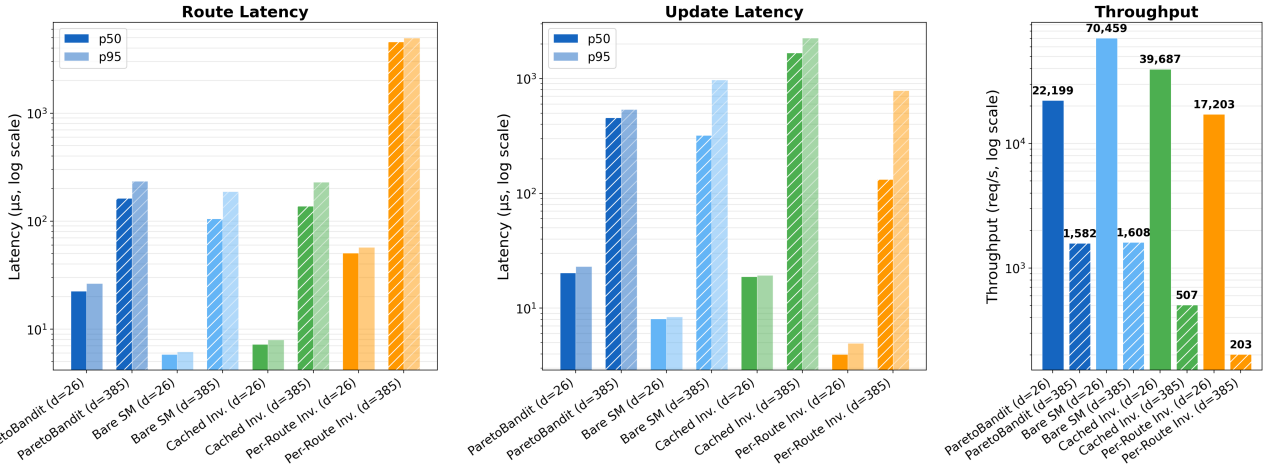


Figure 13. Route latency (left), update latency (centre), and throughput (right) for eight router configurations on a log scale. Bare SM and Cached Inv. share identical `route()` code, so the route panel confirms they are equivalent; the update panel exposes Sherman-Morrison's actual contribution ($O(d^2)$ vs. $O(d^3)$). ParetoBandit bars quantify the additional cost of production features.

execute literally the same `route()` method (UCB scoring via cached A_a^{-1} and $\hat{\theta}_a$), so any route-latency difference is measurement noise.

Route latency is near-identical between Bare SM and Cached Inv. at each dimension (5.8 vs. 7.2 μs at d=26; 105.2 vs. 138.1 μs at d=385), consistent with their shared `route()` code path, with the small residual gap likely reflecting Python object-layout differences between the two subclasses. Update latency is where SM helps: at d=385, the $O(d^2)$ SM update is 5.0× faster than full inversion (346.6 μs vs. 1735.8 μs, p50); at d=26, the gap narrows to 2.3× (8.1 μs vs. 18.6 μs) because $d^3 = 17{,}576$ is already fast for LAPACK. Per-Route Inv. never caches A_a^{-1}, paying K full inversions on every `route()` call; no reasonable implementation would do this, but it bounds worst-case behaviour.

Production overhead. ParetoBandit is the full production router. It uses the same SM update as Bare SM but adds a threading lock around `select_arm()` and `update()`, budget pacing, forgetting-factor decay, staleness tracking,

Table 11. End-to-end latency breakdown for ParetoBandit's production CPU pipeline. Reported values are p50 and p95 over 200 measured iterations after a 50-iteration warmup; percentages are computed relative to the p50 total.

Stage	p50	p95	% of total
Embed (MiniLM-L6-v2, CPU)	8.8 ms	9.3 ms	98.0%
PCA + whitening	0.11 ms	0.15 ms	—
`route()`	0.056 ms	0.072 ms	0.6%
Total E2E (CPU)	**9.0 ms**	**9.4 ms**	100%

and constraint filtering. Comparing ParetoBandit with Bare SM isolates the cost of these production features.

Key findings. **Production overhead is negligible.** At the production dimension (d=26), ParetoBandit completes a full route + update cycle in 43.0 μs (p50), adding $<0.05\%$ latency even to the fastest LLM call ($\sim$100 ms time-to-first-token for Llama-3.1-8B). The router sustains $\approx$22,000 req/s on a single CPU core, sufficient for all but the most extreme throughput tiers.

SM's benefit is in update cost, not routing. Bare SM and Cached Inv. have near-identical route latency; the improvement appears in update: at d=385, SM is 5.0$\times$ faster (346.6 μs vs. 1735.8 μs, p50), and 2.3$\times$ faster at d=26.

Production features add bounded overhead. At d=26, ParetoBandit's route is 3.9$\times$ and update 2.5$\times$ slower than Bare SM due to locking and pacing; at d=385, these ratios shrink (1.7$\times$ route, 1.4$\times$ update) because linear algebra dominates fixed overhead.

PCA yields a $\approx$14.8$\times$ throughput gain. Within production configs, reducing d from 385 to 26 improves throughput from 1,487 to 21,983 req/s ($\approx$14.8$\times$), consistent with $O(d^2)$ scaling plus fixed costs.

End-to-end latency and comparisons. Table 11 breaks down ParetoBandit's production CPU pipeline (MiniLM-L6-v2 embedding + PCA + `route()`) on a single Apple M-series core (200 iterations, 50-iteration warmup).

The embedding step dominates at 97.9% of total time (9.6 ms p50), PCA + whitening adds 0.18 ms, and `route()` contributes 0.080 ms (0.8% of total), for a 9.8 ms end-to-end latency (p50). The routing algorithm is not the bottleneck; prompt embedding is, and every contextual router (e.g., RouteLLM, CSCR, LLM Bandit) faces the same floor.

Among published routers, PROTEUS (Bhatti et al., 2026) reports 8.7 ms p50 per-query latency on an A100 GPU, dominated by a DeBERTa-v3-small encoder forward pass. ParetoBandit matches this class of latency on CPU only (9.8 ms), with the routing decision itself (22.5 μs at d=26) $\approx$390$\times$ faster than PROTEUS's per-query forward pass. Other routing systems—LLM Bandit (Li, 2025), BaRP (Wang et al., 2025a), MixLLM (Wang et al., 2025b), RouteLLM (Ong et al., 2025), HybridLLM (Ding et al., 2024), OmniRouter (Mei et al., 2025), Lookahead (Sun et al., 2025)—do not report per-decision latency.

The vLLM Semantic Router (Liu et al., 2026), a system-level domain/safety classifier, reports 9–22 ms routing on MI300X GPUs depending on signal type, with embedding-based signals dominating latency. CSCR (Shirkavand et al., 2025) claims "microsecond" k-NN lookups but excludes the encoder forward pass. Infrastructure-layer proxies such as Portkey Gateway (Portkey AI, 2025) ($<$ 1 ms) and Kong AI Gateway (Acquaviva, 2025) (24 ms p95) provide a lower bound on non-routing overhead. Figure 14 places these systems and ParetoBandit on a common log-scale alongside typical LLM inference latencies, showing that all routing overheads are orders of magnitude below model serving time.

Routing overhead relative to LLM inference. Table 12 contextualises ParetoBandit's 9.8 ms E2E overhead against the inference latency of the paper's own K=4 model portfolio, measured via 20 streaming API calls per configuration through OpenRouter.

Table 12. Routing overhead as a fraction of LLM inference latency (K=4 portfolio, 20 streaming API calls via OpenRouter). *TTFT* = time to first token. Routing overhead is the 9.0 ms end-to-end CPU critical path (embedding + PCA + `route()`).

LLM	Prompt	TTFT (ms)	Total (ms)	Routing / Total
Llama-3.1-8B	short	820	7,001	0.13%
Llama-3.1-8B	medium	607	9,958	0.09%
Llama-3.1-8B	long	684	7,235	0.12%
Mistral-Large-2512	short	1044	5,811	0.16%
Mistral-Large-2512	medium	665	6,442	0.14%
Mistral-Large-2512	long	636	8,445	0.11%
Gemini 2.5 Flash	short	758	2,574	0.35%
Gemini 2.5 Flash	medium	858	3,256	0.28%
Gemini 2.5 Flash	long	890	3,756	0.24%
Gemini 2.5 Pro	short	6508	6,617	0.14%
Gemini 2.5 Pro	medium	7901	8,068	0.11%
Gemini 2.5 Pro	long	8188	8,638	0.10%

Even against the fastest model and shortest prompt (Gemini 2.5 Flash, 2,574 ms total latency), the full routing pipeline adds $\approx$0.38% overhead; for the slowest model (Gemini 2.5 Pro, long prompt, 8,638 ms), routing contributes $\approx$0.11%. The routing decision is effectively invisible to the end user.

G Recovery Limit under Quality Degradation

Experiment 03 (§4.4) showed that the system recovers from a silent quality regression (Mistral-Large reward dropping to 0.75) within a 608-prompt Phase 3 horizon. Here we extend that result across a range of severities. The system trends toward recovery at all tested levels—including near-total degradation—but deeper corruption requires more Phase 3 prompts under normal conditions to fully converge, and our evaluation horizon is too short to observe complete recovery at the most extreme levels. We therefore characterise a

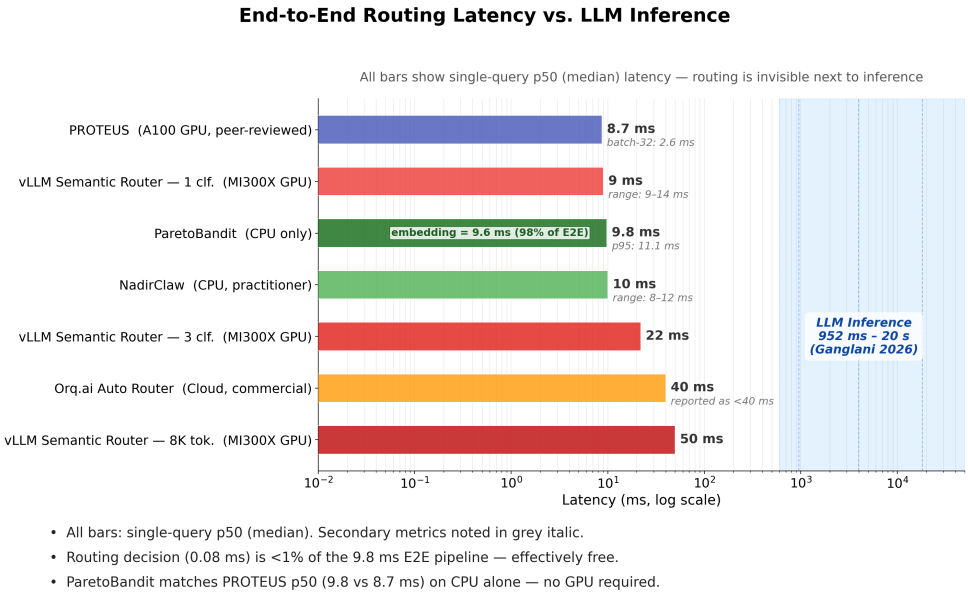


Figure 14. End-to-end routing latency (p50, single query) for published routing systems vs. LLM inference time (shaded region).

finite-horizon recovery envelope: for a given Phase 3 length, the degradation severity up to which the system reaches near-Phase 1 performance, and how extending the horizon shifts that boundary.

Setup. We sweep Mistral-Large's degraded reward from 0.05 to 0.85 (normal reward ≈ 0.92). Degradation severity, the x-axis in Figure 15a, is the fractional gap between the degraded reward and the Phase 1 system baseline (≈ 0.89), ranging from $\approx 4\%$ (mild) to $\approx 94\%$ (near-total). Degradation is modelled as a mean shift: per-prompt rewards are shifted so the degraded arm's mean equals the target level while retaining prompt-dependent variation (clipped to $[0, 1]$), which is more realistic than constant-reward replacement. Cost is held fixed during Phase 2 so only the reward signal reveals the regression. All runs use the moderate budget ($\$6.6\times10^{-4}$ per prompt), 20 seeds, and the same hyperparameters as the main experiment. We define full recovery as a Phase 3/Phase 1 reward ratio $\geq 97\%$.

For the extended-horizon evaluation, Phase 3 uses all non-Phase-2 holdout prompts (1,216 prompts, $2\times$ the base horizon) without recycling, so the i.i.d. arrival assumption is preserved and extended-horizon results reflect recovery on fresh prompts.

Recovery mechanisms. Recovery operates through two mechanisms. Geometric forgetting ($\gamma<1$) exponentially down-weights corrupted Phase-2 observations so that fresh Phase-3 data dominates the mean estimate; this is the primary driver. Staleness-driven exploration inflates UCB variance for non-selected arms and can trigger re-exploration, but under budget-constrained routing the cost penalty can exceed the exploration bonus, limiting this effect on its own. Both mechanisms act jointly, but the forgetting half-life (≈ 231 steps at $\gamma=0.997$) sets the characteristic timescale.

Finite-horizon recovery envelope (Figure 15a). Figure 15a plots the P3/P1 reward ratio against degradation severity for two Phase 3 lengths. At the 608-prompt horizon (solid blue), full recovery ($\geq 97\%$) is achieved for degradations up to $\approx 17\%$, after which the ratio declines smoothly to a floor of $\approx 90\%$. Doubling Phase 3 to 1,216 unique prompts (dashed orange) uniformly lifts the curve: the 97% boundary shifts to $\approx 30\%$ degradation, and the severe-degradation floor rises to $\approx 93\%$.

This uniform lift is consistent with the floor being a finite-horizon artefact rather than a fundamental limit on recoverable severity: geometric forgetting continues erasing Phase-2 corruption, so more Phase-3 prompts yield more recovery. Within any fixed horizon, severity beyond $\approx 50\%$ has diminishing marginal impact because the forgetting dynamics are unchanged; only the initial bias differs. At the other extreme, mild degradation ($\leq 10\%$) can produce P3/P1 ratios above 100%: Phase 2 drives the policy away from the degraded arm, and for some prompts the alternative routing discovered during degradation is genuinely better than the original policy.

Convergence dynamics (Figure 15b). The extended-horizon time series (50-prompt rolling window) confirm this interpretation. All trajectories rise monotonically toward the Phase 1 baseline with no sign of plateauing, but convergence speed is severity-dependent: milder degradations reach baseline well before 1,216 prompts, while more severe degradations are still improving at the horizon. In all cases the confidence bands narrow over time, indicating sta-

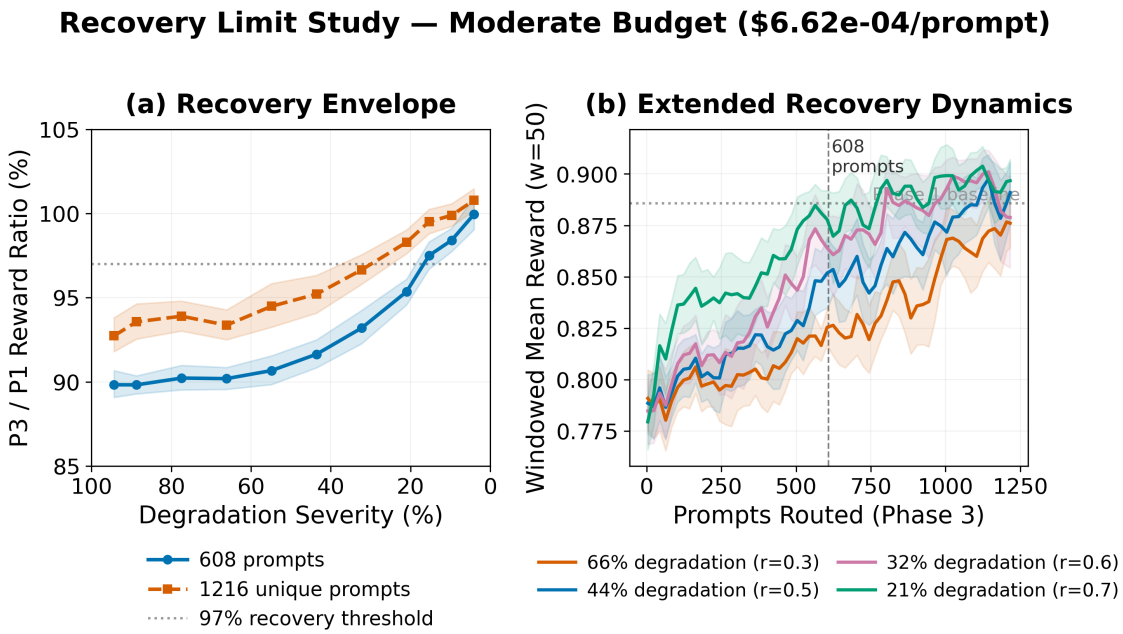


Figure 15. Recovery limit under quality degradation (moderate budget, 20 seeds, 95% bootstrap CI). **(a)** P3/P1 reward ratio vs. degradation severity; dotted line = 97% full-recovery threshold. **(b)** Phase-3 recovery dynamics (50-prompt rolling window) at the extended horizon for four severity levels.

ble convergence rather than high-variance oscillation. The monotonic, non-plateauing trajectories are consistent with recovery being rate-limited by Phase-3 length rather than fundamentally bounded by degradation severity, though we cannot confirm full convergence at the most extreme levels within our evaluation budget.

Limitations and deployment implications. This study evaluates a single budget level and a single-arm, quality-only degradation; other budgets or simultaneous multi-arm regressions may behave differently. For typical silent regressions ($\leq$20%), ParetoBandit recovers automatically within the 608-prompt observation window. For more severe regressions, recovery still proceeds but requires a longer Phase 3 horizon; operators can either allow additional routing time or intervene with an explicit model demotion to accelerate convergence.